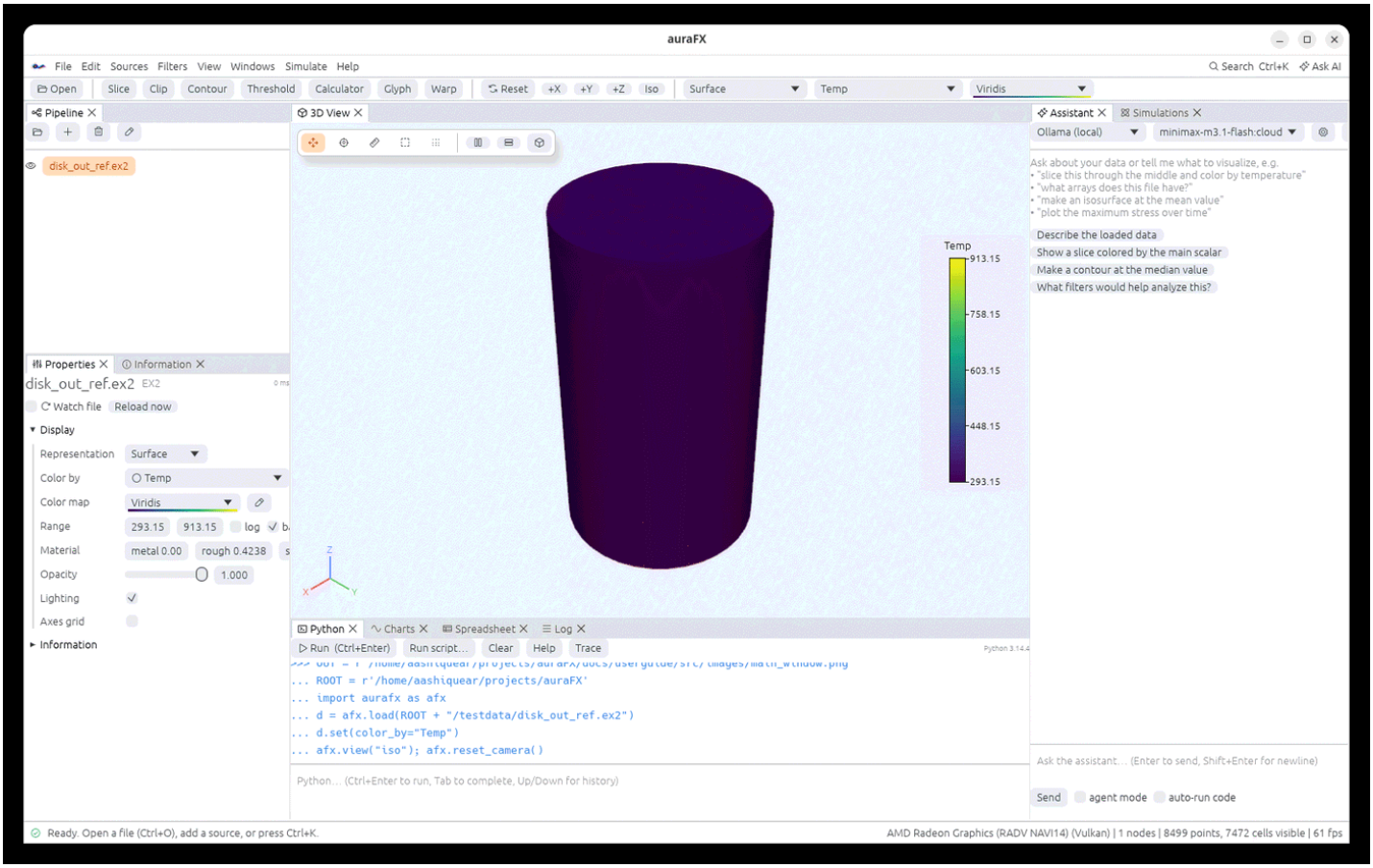
auraFX User Guide

auraFX is a scientific visualization, analysis and simulation workbench. It opens the files your solvers write (Exodus, VTK, OpenFOAM, LAMMPS, CGNS, EnSight, NetCDF and many more), shows them in a 3D view drawn by your GPU, lets you cut, contour, threshold, trace and measure them with the usual filters, and adds three things next to the 3D view:

- a **Python console** with a one-line-per-idea API (`afx.load("run.e").slice().color_by("T")`),
- an **AI assistant** that sees your pipeline and writes runnable code for you,
- a **simulation runner** that launches solvers locally, over SSH or through Slurm and opens the results when they finish.

This guide walks through every feature. Each chapter says where a feature is in the window, how to call it from Python, and what its limits are.



How this guide is organised

Part	What it covers
Basics	opening files, the pipeline, filters, coloring, views, selection, time and animation
Rendering	lighting, shadows, ambient occlusion, ray tracing, materials, volumes, point clouds
Scripting	the Python API, the AI assistant, writing plugins
Data	every file format read and written, exporting pictures and scenes, running simulations, working on remote machines
Reference	settings and keyboard shortcuts

Formats of this guide

The guide is kept as Markdown in the repository (`docs/userguide/src`), published as an online book, and printed to a PDF (`docs/userguide/auraFX-user-guide.pdf`). All three come from the same source files, so they always say the same thing.

Getting started

Installing

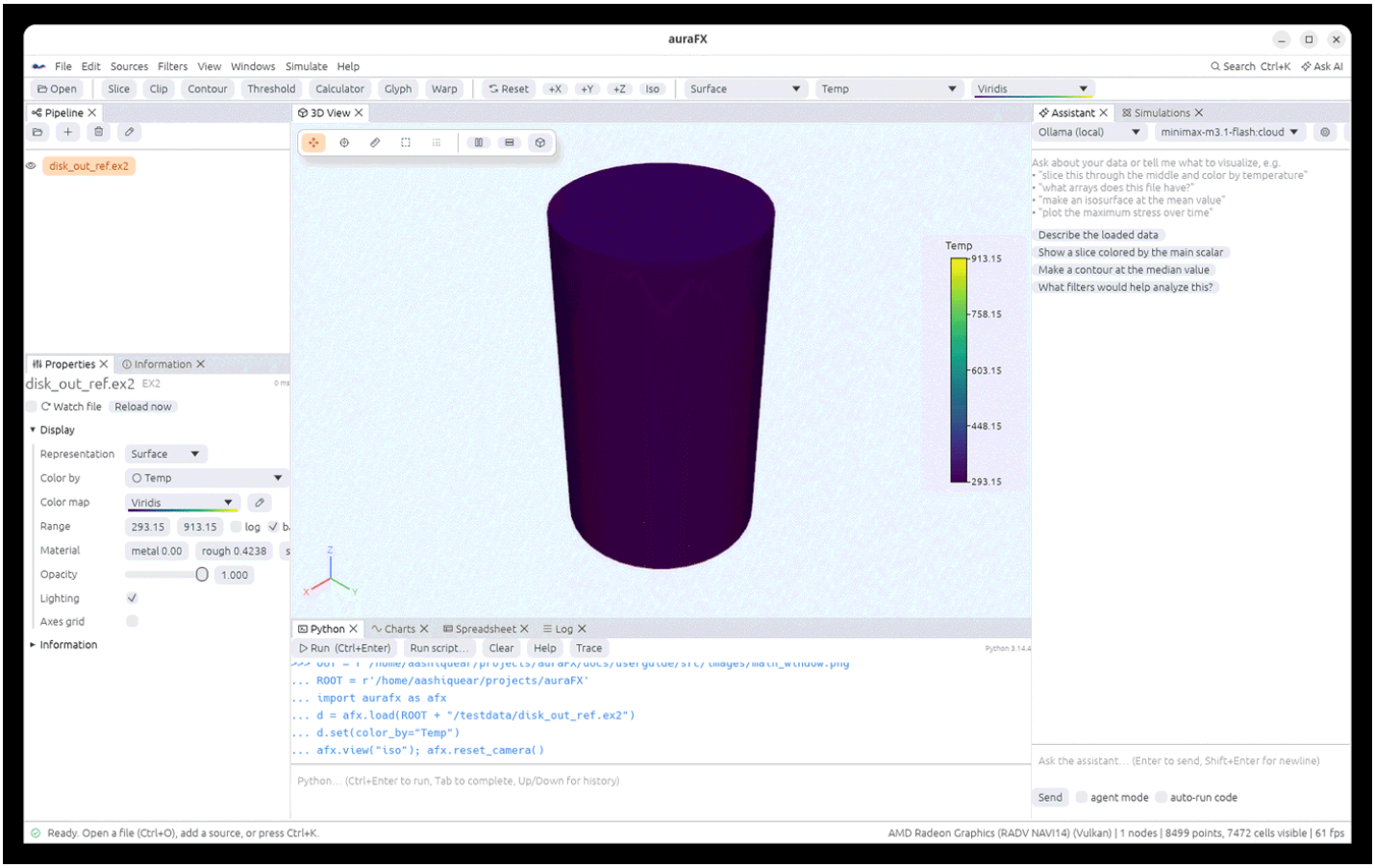
Prebuilt packages for Linux, macOS and Windows are attached to each release, and a one-line installer puts the app in your path:

```
curl -fsSL
https://raw.githubusercontent.com/aashiquear/auraFX/main/scripts/install.sh | sh
# Linux, macOS
irm https://raw.githubusercontent.com/aashiquear/auraFX/main/scripts/install.ps1 |
iex          # Windows PowerShell
```

Desktop packages bundle Python 3.12 with NumPy, so the Python console works without setup. To build from source you need Rust 1.85 or newer, Python 3.10+ with development headers, and a Vulkan (Linux, Windows) or Metal (macOS) GPU driver:

```
cargo build --release
./target/release/aurafx                    # start
./target/release/aurafx run.e              # open a file
./target/release/aurafx --script setup.py # run a Python script at startup
```

The window



The window has five areas; every panel can be closed, reopened from the **Windows** menu, and dragged to another place.

Area	Purpose
Pipeline (top left)	the files you opened and the filters below them; click to select, click the eye to show or hide
Properties / **Information** (bottom left)	the selected object's parameters and display settings; the Information tab shows arrays, ranges, bounds, cell types and time steps
3D View (middle)	the picture; drag to rotate, right-drag or wheel to zoom, middle-drag (or Shift-drag) to pan, double-click to fit
Python, **Charts**, **Spreadsheet**, **Log** (bottom)	the console, plots, the data as a table, messages
Assistant, **Simulations** (right)	the AI chat and the job runner

The toolbar has the most used filters (Slice, Clip, Contour, Threshold, Calculator, Glyph, Warp), the camera buttons (Reset, +X, +Y, +Z, Iso) and, for the selected object, its representation, the

array it is colored by and the color map. The status bar at the bottom names the GPU in use and counts the visible points and cells.

Opening a file

- **File › Open…** (Ctrl+O), or drop a file onto the window, or start the app with the file name.
- **File › Open Recent** keeps the last files; the three sample datasets at the bottom of that menu (a heated disk, a wavelet volume, a sphere) are good for trying things out.
- Numbered files (`run_0001.vtu`, `run_0002.vtu`, …), `.pvd` collections, Exodus and OpenFOAM cases open as time series: the time toolbar appears under the main toolbar.

When a file opens, the object is colored by its first array and the camera fits the data. The file is watched: if a solver rewrites it, auraFX reloads it (**Watch file** in Properties).

```
d = afx.load("run.e")              # Python does the same
d.set(color_by="Temp")
afx.view("iso"); afx.reset_camera()
```

Applying a filter

Select an object, then pick a filter from the toolbar, the **Filters** menu, or the command palette (**Ctrl+K**, type part of its name). The filter runs at once with sensible defaults (a slice through the center, a contour at the middle value) and appears below its input in the Pipeline. Change its parameters in Properties; they apply as you type. **Re-run** forces a recomputation and **Reset to defaults** restores the defaults.

Filters can be stacked: a Contour of a Clip of a file. Delete an object with the trash button or the Delete key; the objects below it go too.

Saving your work

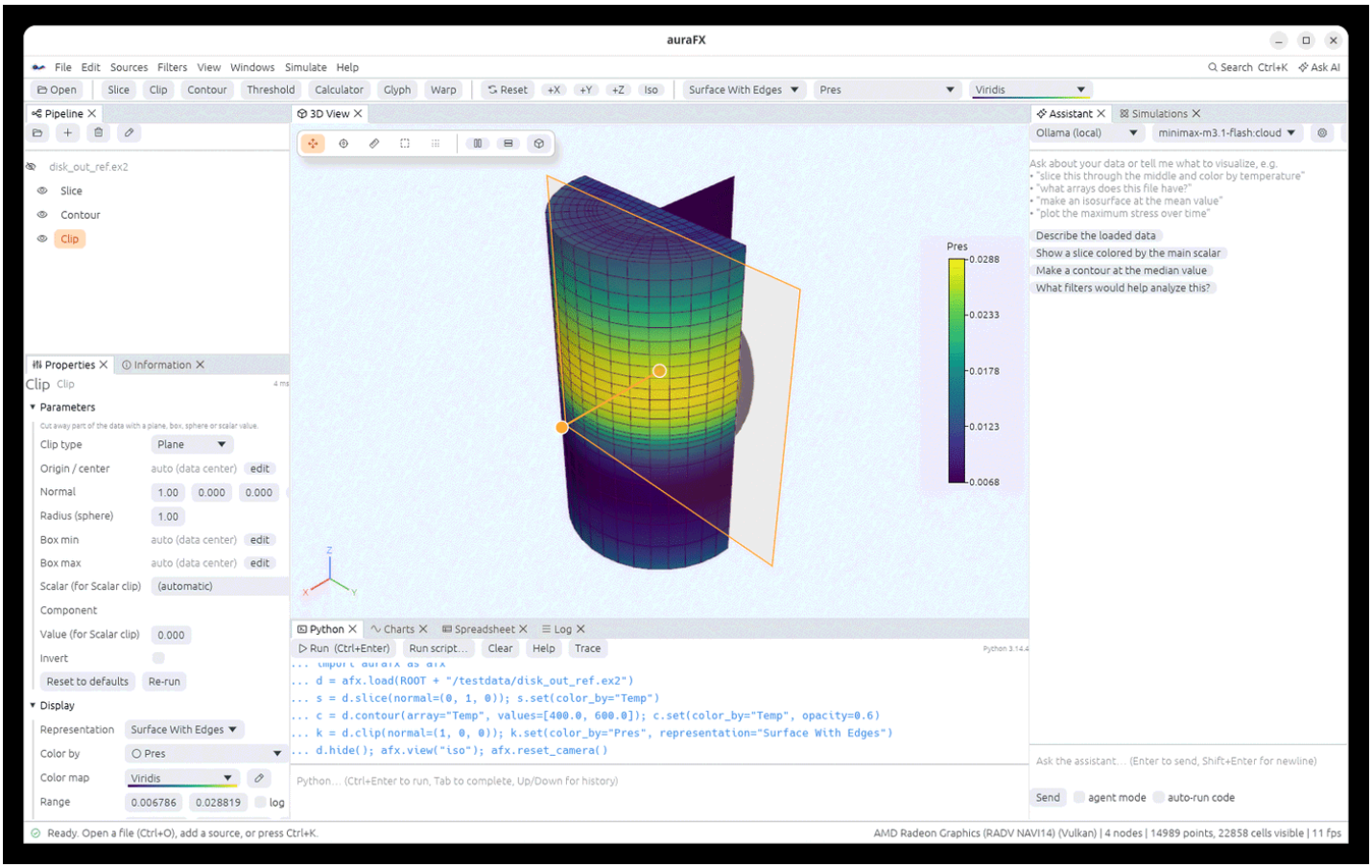
- **File › Save State** (Ctrl+S) writes an `.afx` file holding the pipeline, every parameter, the display settings, the camera and any keyframes. **File › Open** reopens it, including objects that live on a remote server.
- On the next start, **Restore last session** in the empty view brings back the last state.
- Right-click an object › **Save data…** writes its output to a file (see File formats).
- **Edit › Undo / Redo** cover every change to the pipeline and display settings.

Getting help

- **Help › Keyboard shortcuts** and **Help › Python API cheat sheet** open inside the app.
- The command palette (Ctrl+K) finds filters, sources and actions by name.
- The AI assistant answers questions about the loaded data and the app (see AI assistant).

Filters

A filter takes the output of the object above it and makes a new object. Select an input, choose the filter, adjust it in Properties. Every filter is also a method in Python (`node.slice(...)`), and the assistant knows all of them.



Common filters (toolbar)

Filter	What it does	Main parameters
Slice	the data on a plane or a sphere, with the values interpolated onto it	type, origin, normal (or radius); the plane can be dragged in the view
Clip	keeps one side of a plane, the inside/outside of a box or sphere, or the cells above/below a scalar value; **crinkle** keeps whole cells	type, origin, normal, box corners, radius, scalar and value, invert

Filter	What it does	Main parameters
Contour	isosurfaces (or isolines on 2-D data) at one or more values	array, values; computes normals
Threshold	the cells whose values lie in a range	array, min, max, all/any point
Calculator	a new array from an expression over other arrays (`sqrt(u*u+v*v)` , `mag(V)` , vectors with `iHat` …)	expression, result name, point or cell
Glyph	arrows, cones, spheres, cubes or lines at the points, scaled and oriented by arrays	shape, orientation and scale arrays, factor, maximum glyphs
Warp By Vector / Scalar	moves points by a vector array (displacements) or along the normal by a scalar	array, factor
Stream Tracer	streamlines of a vector field from a line or a point cloud of seeds; **Tube** makes them solid	vectors, seed type, seed line or cloud, integration direction and length
Plot Over Line	samples the data along a line and plots it in Charts	the two end points (draggable)
Probe Location	the values at one point, shown in Information and the Spreadsheet	location (draggable)

Analysis

Resample To Image, Resample With Dataset, Histogram, Descriptive Statistics, Integrate Variables (areas, volumes and integrals of every array), Gradient (with divergence, vorticity and Q-criterion), Cell Size, Mesh Quality, Elevation, Cell Centers, Connectivity, Find Data, Extract Selection, Temporal Statistics (mean, min, max and standard deviation over all time steps), Temporal Interpolator, Temporal Shift Scale, Temporal Array Operator, and the particle filters (Particle Tracer, Particle Path, Streak Line) that follow particles through a time-varying flow.

Geometry

Extract Surface, Extract Edges, Feature Edges, Outline, Transform, Reflect, Shrink, Clean, Surface Normals, Smooth, Triangulate, Tetrahedralize, Tessellate (curved cells into straight pieces),

Mask Points, Merge Blocks, Extract Block, Extract Subset (a sub-volume of an image), Append Datasets / Geometry / Attributes, Cell Data to Point Data and back, Extract Component, Rename Array, Pass Arrays, and OpenFOAM Boundary Patches.

Sources

Sources adds objects without a file: Sphere, Cone, Cylinder, Box, Plane, Line, Point Cloud, Wavelet (a test volume) and Unstructured Cell Types (one of every cell type, for tests).

Programmable filters

Programmable Filter, **Programmable Source** and **Python Calculator** run Python that is stored in the pipeline and in state files. The filter's code gets `data` (points and arrays as NumPy) and returns it changed, or a new dataset built with `afx.make_*`. For code you want to keep across sessions, write a plugin.

Filters with several inputs

Append Datasets, Append Geometry, Append Attributes and Resample With Dataset take several inputs. Ctrl+click the inputs in the Pipeline, then choose the filter; the Properties panel gets an **Inputs** section where the order can be changed.

```
afx.apply_multi("append_datasets", [a, b, c])
r = probe_mesh.resample_with_dataset(source)
```

Parameters that say "auto"

Many parameters start as **auto (data center)** or similar: the value is taken from the data each time it runs (the center of the bounds, the middle of a range, 5 % of the size). Press **edit** to type a value instead. Auto values follow the data through time steps and across pieces of a parallel server.

Speed

Slice, Contour, Clip, Threshold, Extract Surface, Stream Tracer, Gradient and Integrate Variables run on every core and give exactly the same result on any number of threads. A slice of 4 million cells takes about 80 ms. The thread count can be limited in **Settings › Filter threads** (0 = all cores) or with `AURAFX_THREADS`.

From Python

```
afx.filters()                 # every filter with its parameters
afx.help_filter("clip")       # one filter explained
s = d.slice(normal=(0, 1, 0))
c = d.contour(array="Temp", values=[400, 600])
k = d.clip(type="Box", box_min=(-3, -3, -2), box_max=(3, 3, 4))
t = d.threshold(array="Pres", min=0.01)
s.set(origin=(0, 1, 0))       # change a parameter; the filter re-runs
s.update()                    # wait for it
```

Coloring and color maps

Color by an array

The **Color by** box in Properties (and in the toolbar) lists every point and cell array of the object. Vectors are colored by magnitude unless a **Component** is chosen. **Solid color** uses one color for the whole object. New files are colored by their first array automatically.

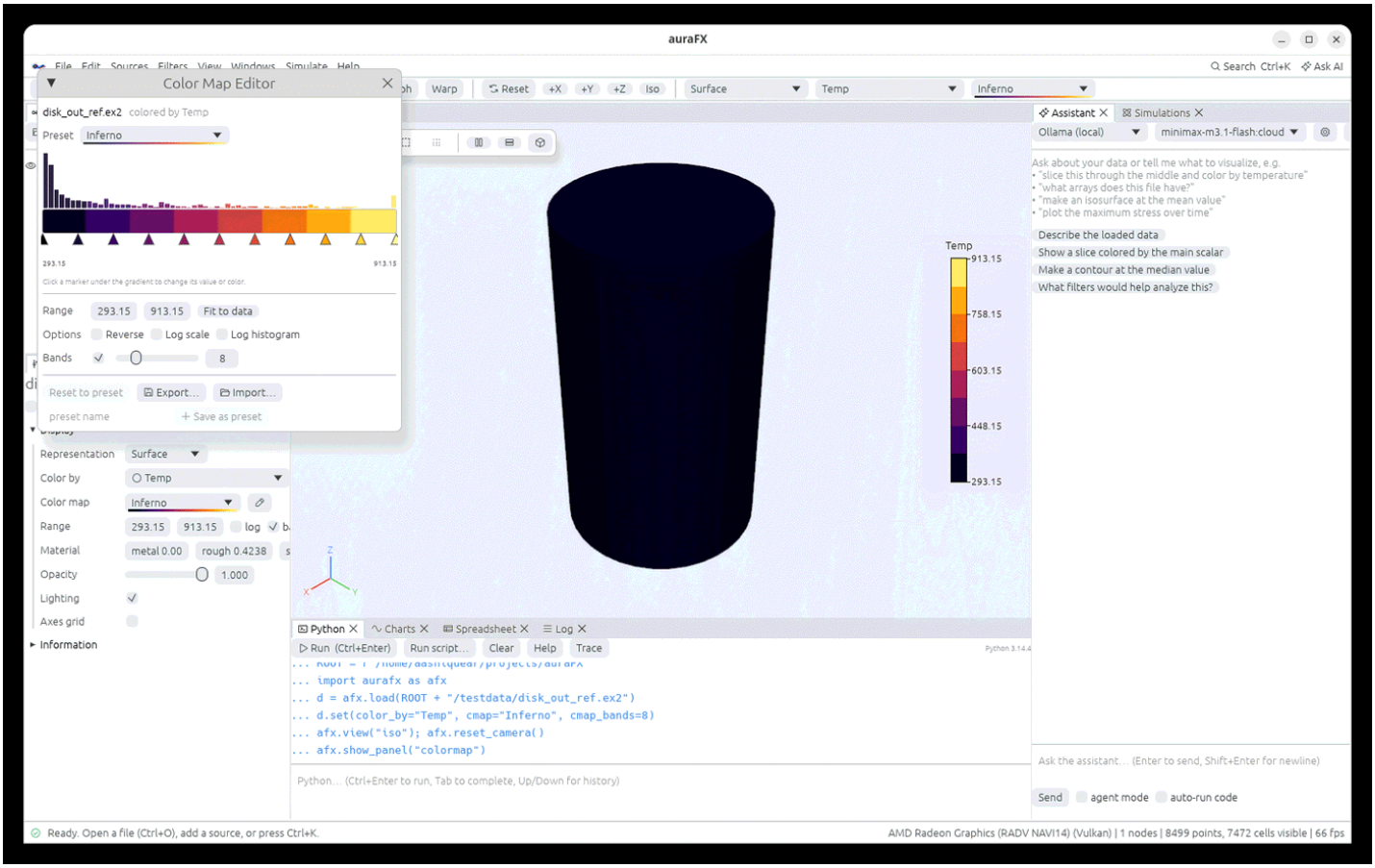
Range is taken from the data; type values to fix it (useful when comparing time steps or objects), press **auto** to return to the data range, tick **log** for a logarithmic scale and **bar** to show the legend in the view. Values that are not a number (NaN, fill values of NetCDF files) are drawn grey.

```
d.set(color_by="Temp", cmap="Inferno", range=(300, 900), log_scale=False)
d.set(color_by="V", component=0)        # x component of a vector
d.set(color_by=None, solid_color=(0.8, 0.2, 0.1))
```

Color maps

Eighteen maps ship with the app (Viridis, Inferno, Magma, Plasma, Cool to Warm, Jet, Rainbow, Grayscale, ColorBrewer sets and more); the **Color map** box shows each with a preview and marks the ones safe for color-blind readers. **Reverse** flips a map and **Bands** turns it into steps.

The color map editor



View › Color Map Editor… (or the pencil next to the map) opens the editor for the selected object:

- the histogram of the array behind the map shows where the values lie;
- drag the markers to move colors, double-click to change a color, right-click to delete one, click between markers to add one;
- **Bands**, **Reverse**, and the range fields as in Properties;
- **Save as preset** keeps your map under a name for other objects and sessions;
- **Import…** reads ParaView color map files (`.json`) as presets; **Export…** writes them.

```
afx.colormaps()                                    # names
afx.define_colormap("hot steel", [[0, 0, 0, 0], [0.5, 1, 0.3, 0], [1, 1, 1, 1]])
afx.import_colormaps("paraview_maps.json")
d.set(cmap="hot steel", cmap_reverse=True, cmap_bands=8)
d.set(cmap_points=[[0, 0, 0, 1], [1, 1, 0, 0]])    # a one-off map on this object
```

The legend

The legend (scalar bar) shows the map, its range and the array name for the selected object, or for the first object with a legend when nothing is selected. Untick **bar** to hide it. It is included in screenshots and animations, and drawn as real text in SVG exports.

Views, layouts and the window

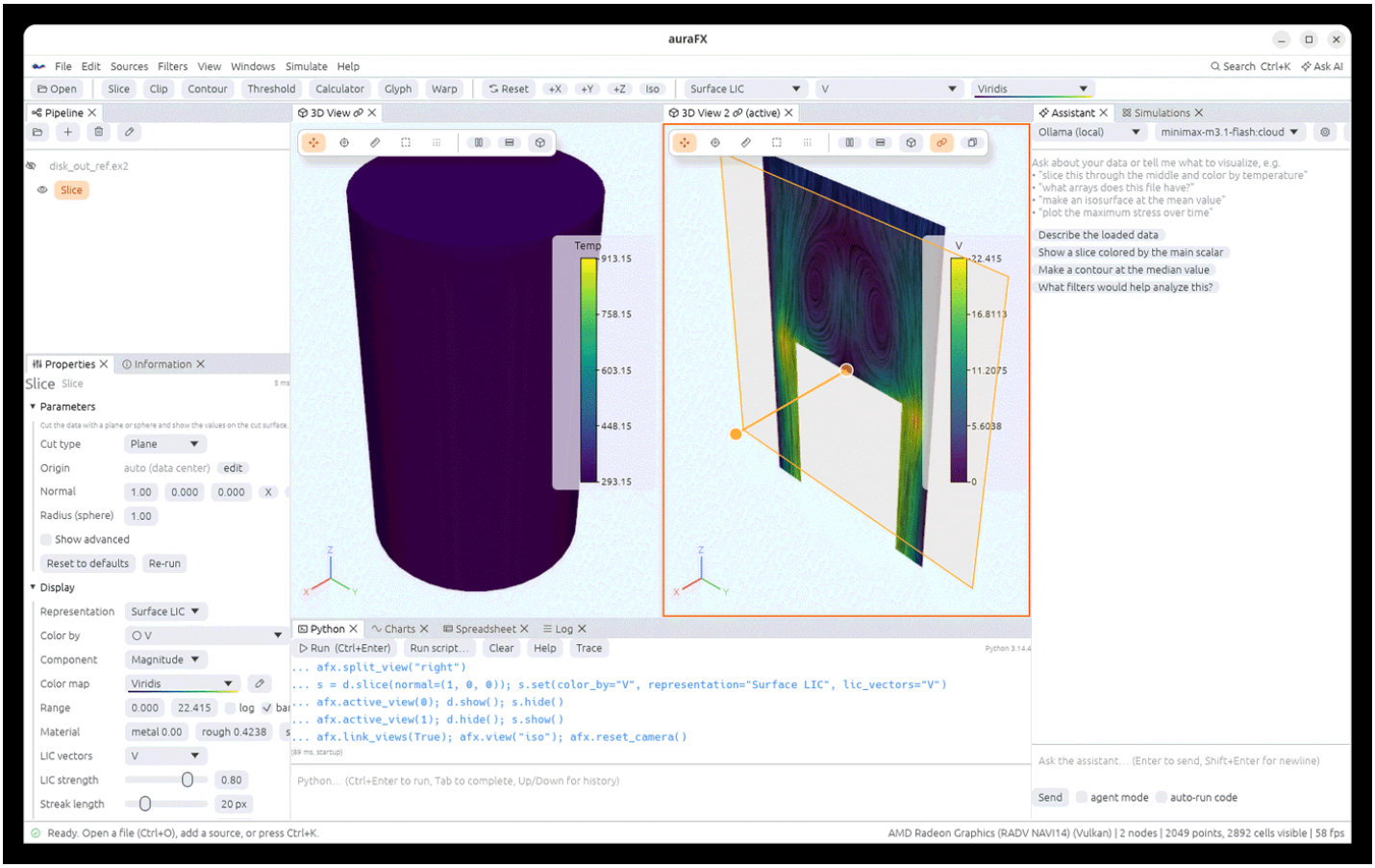
The 3D view

Action	Mouse / key
rotate	drag
pan	middle-drag, or Shift+drag
zoom	wheel, right-drag, or Ctrl+drag; pinch on a trackpad
fit the data	double-click, **R**, or the **Reset** button
look along an axis	**+X**, **+Y**, **+Z**, **Iso** in the toolbar, or **View › Look from…**
orthographic projection	**View › Orthographic projection**
orientation axes (bottom left)	**View › Orientation axes**

The small toolbar inside the view switches the mouse mode: navigate (**Esc**), pick (**P**), ruler (**M**), select cells (**S**) and select points (**D**); see Selection, picking and widgets.

```
afx.view("iso"); afx.view("+x"); afx.view([1, 0.5, 0.3])   # a direction from the
data to the camera
afx.reset_camera()
```

Several views



The split buttons in the view toolbar (or **Windows › Split view right / down**, `afx.split_view("right")`) add a 3D view next to the active one. Each view has its own camera and its own list of visible objects; click a view to make it active (orange frame), then the eye icons in the Pipeline and `node.show()` / `hide()` apply to it. From Python, `afx.active_view(1)` makes a view active (0 is the first one) and returns once it is, so the lines after it act on that view:

```
afx.split_view("right")                  # returns the new view's number (1)
afx.active_view(0); d.show(); s.hide()    # left: the data
afx.active_view(1); d.hide(); s.show()    # right: the slice
```

- **Link** (the chain button, `afx.link_views(True)`) keeps the cameras of the linked views together; **Match** (`afx.match_views()`) copies the active camera to the others once.
- Right-click an object › **Send to view** moves it to another view or into a new one (`node.send_to_view(1)` or `"right"`, `keep=True` to keep it in the current view too).
- Close a view with the × on its tab; its objects stay in the pipeline.

Panels and layouts

The **Windows** menu lists every panel (Pipeline, Properties, Information, 3D views, Python, Charts, Spreadsheet, Log, Assistant, Simulations) with a tick to show or hide it, and four layout presets:

Preset	Shows
Visualize	pipeline, properties and a large 3D view
Analyze	the view with charts and the spreadsheet
Script	the view with a tall Python console
Simulate	the simulations panel, the log and the view

Reset layout returns to the default. Panels can be dragged by their tabs into new positions or into tabs of each other; the arrangement is remembered between sessions.

```
afx.layout("analyze")
afx.show_panel("Charts"); afx.hide_panel("Log")
```

Theme and scale

Settings › Theme chooses dark, light or follow-the-system; the 3D background follows the theme (and can be set per theme). **UI scale** enlarges the whole interface. From Python: `afx.set_theme("light")`.

Command palette

Ctrl+K (or Ctrl+Space, or the **Search** button) opens the palette: type a few letters of a filter, a source, a panel or an action (open, save screenshot, reset camera, undo, light theme, ask the AI…) and press Enter. It is the fastest way to reach anything in the app.

Undo, redo and state

Edit › Undo / Redo (Ctrl+Z, Ctrl+Y) cover opening files, adding and deleting filters, parameter and display changes. **File › Save State** writes everything to an `.afx` file (see Getting started); `afx.save_state("work.afx")` and `afx.load_state("work.afx")` do the same from Python.

Selection, picking and widgets

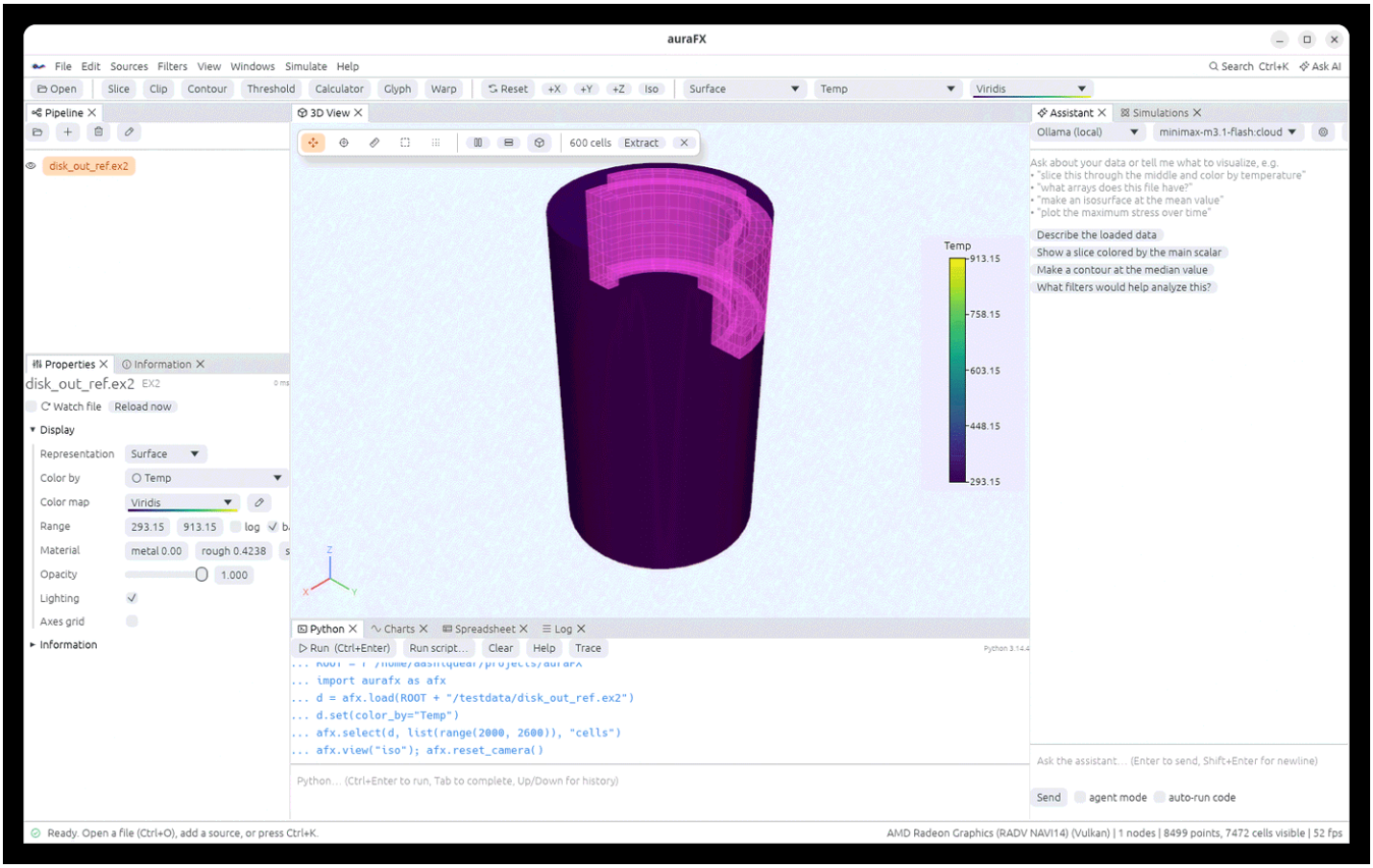
Picking values

Press **P** (or choose the pick mode in the view toolbar, or Ctrl+click in any mode) and click a surface: the point is marked and the Information panel lists every array's value there, with the object's name. `afx.probe(node, (x, y, z))` returns the same values from Python.

The ruler

Press **M**, click two points on surfaces: the distance is drawn and printed between them. Press M again (or Esc) to leave the mode.

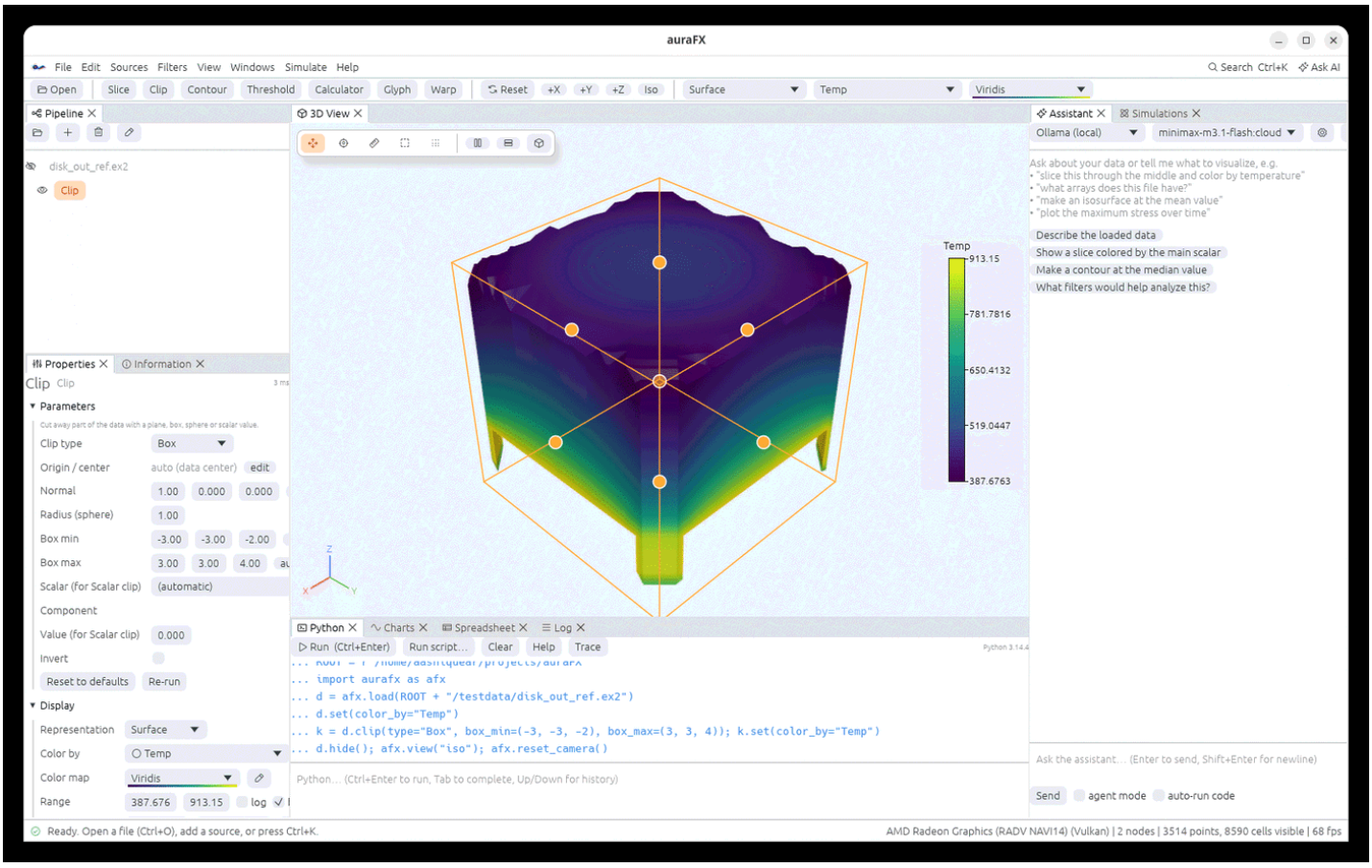
Selecting cells and points



Press **S** (cells) or **D** (points) and drag a rectangle over the view. The selected cells are drawn highlighted through the surface. **Shift** adds to the selection, **Ctrl** removes from it. By default only the cells you can see are selected; **Select through** in the view toolbar takes every cell inside the rectangle, hidden ones included.

- **Create an Extract Selection node** (the button in the view toolbar, `afx.extract_selection()`) turns the selection into a new object you can filter further.
- **Find Data** (Filters) selects by a query such as `Temp > 600` or the N largest values.
- The Spreadsheet shows the selected rows; `afx.selection()` returns them, `afx.select(node, ids, "cells")` sets them, `afx.clear_selection()` clears.

Widgets: dragging filter shapes in the view



When the selected filter has a shape, it is drawn in the view with orange handles:

Filter	Widget
Slice, Clip (plane)	a plane: drag the center along the normal, drag the tip to tilt

Filter	Widget
Clip (box), Box source	a box: drag the center to move it, a face handle to resize that side
Slice/Clip (sphere), Stream Tracer point-cloud seeds, Sphere source	a sphere: center and radius handles
Plot Over Line, Stream Tracer line seeds, particle tracers, Line source	a line: either end, or the middle to move both
Probe Location	a point

The parameters in Properties follow the drag, and typing values moves the widget. Custom lights can be aimed the same way; see Lighting.

Annotations

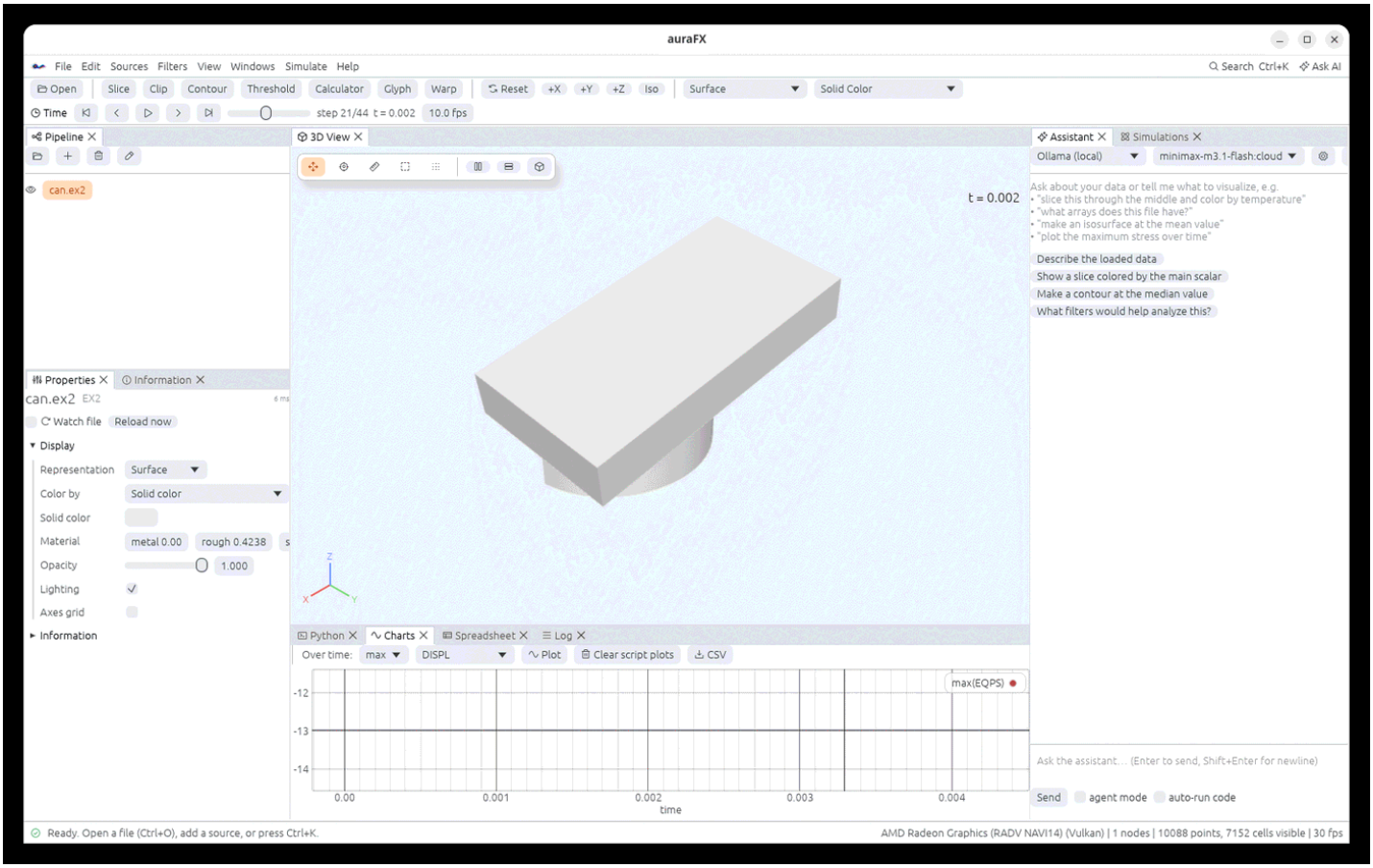
Filters › Annotation, `afx.annotate("Run 42", corner="upper left", size=18)` puts text in a corner of the view; `afx.time_label(True)` (or **View › Time label**) shows the current time. Both are drawn in screenshots and animations. `afx.clear_annotations()` removes them.

Time, charts and animation

Time steps

Files with several time steps (Exodus, `.pvd`, OpenFOAM cases, LAMMPS dumps, numbered file series, VTKHDF, EnSight, Xdmf, NetCDF, CGNS) show the **time toolbar**: play/pause, first, previous, next, last, a slider, the step number, the time value and the playback speed. Every filter follows the time step; filters that look across time (Temporal Statistics, Particle Tracer, Temporal Interpolator…) read the steps they need.

```
afx.times()          # the time values
afx.time(20)         # go to step 20
```



Charts

The **Charts** panel plots lines with zoom, legends and log axes:

- **Plot Over Line** (a filter) plots the arrays along a line;
- **Edit › Plot Over Time…** or `afx.over_time(node, "EQPS", stat="max")` plots a statistic (max, min, mean, sum, or the value of a global variable) over every time step;
- `afx.plot(x, y, label="T")` plots your own NumPy arrays; `afx.clear_plots()` empties the panel.

The **Spreadsheet** panel shows the selected object's points, cells or field data as a table with sorting, column choice and export to CSV.

Animation

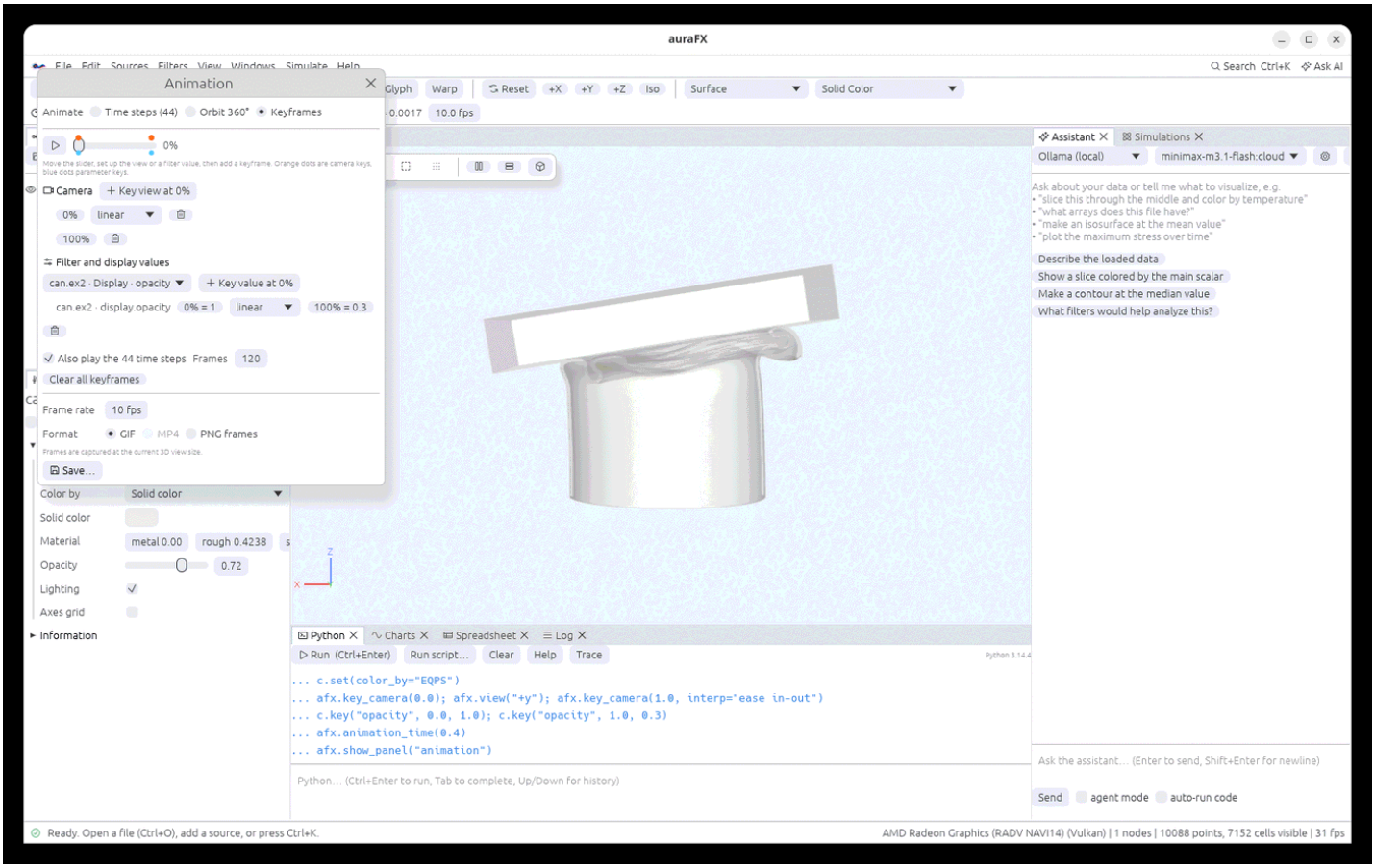
File › Animation… records one of three things:

Mode	What it plays
Time steps	every time step of the data
Orbit 360°	the camera turning once around the data (choose the number of frames)
Keyframes	the keyframe timeline described below

Pick the frame rate and the format: **GIF**, **MP4** (needs `ffmpeg` on the PATH) or a folder of **PNG frames**. Frames are captured at the 3D view's size, with the legend, labels and annotations.

```
afx.save_animation("run.gif", fps=10)          # time steps (or keyframes if any)
afx.save_animation("turn.mp4", orbit=72)       # a turntable
```

Keyframes



The **Keyframes** tab of the Animation window is a timeline from 0 % to 100 %:

1. Move the slider, set up the view, press **Key view at …** to record the camera.
2. Select a filter or display value in the list (any numeric filter parameter, or opacity, point size, line width, colors…), set it, press **Key value at …**.
3. Each key has an interpolation menu for the way it moves on to the next key: **linear**, **step** (hold), **ease in**, **ease out**, **ease in-out**, **exponential** (constant ratio, for zooms and scales) or **wave** (there and back, a sinusoid).
4. Press play to preview; tick **Also play the time steps** to run the data's time along the animation; set the number of **Frames**; **Save…** records it.

The camera moves on a sphere around the focal point, so turns look even and zooms do not rush. Orange dots on the timeline are camera keys, blue dots value keys; click a key to jump to it, right-click to delete it.

```
afx.key_camera(0.0); afx.view("+y"); afx.key_camera(1.0, interp="ease in-out")
c.key("opacity", 0.0, 1.0); c.key("opacity", 1.0, 0.3)
s.key("origin", 0.5, (0, 2, 0), interp="wave")
afx.animation(frames=120, time=True)
afx.animation_time(0.4)                          # preview a moment
afx.save_animation("keys.mp4", fps=24)
afx.clear_keyframes()
```

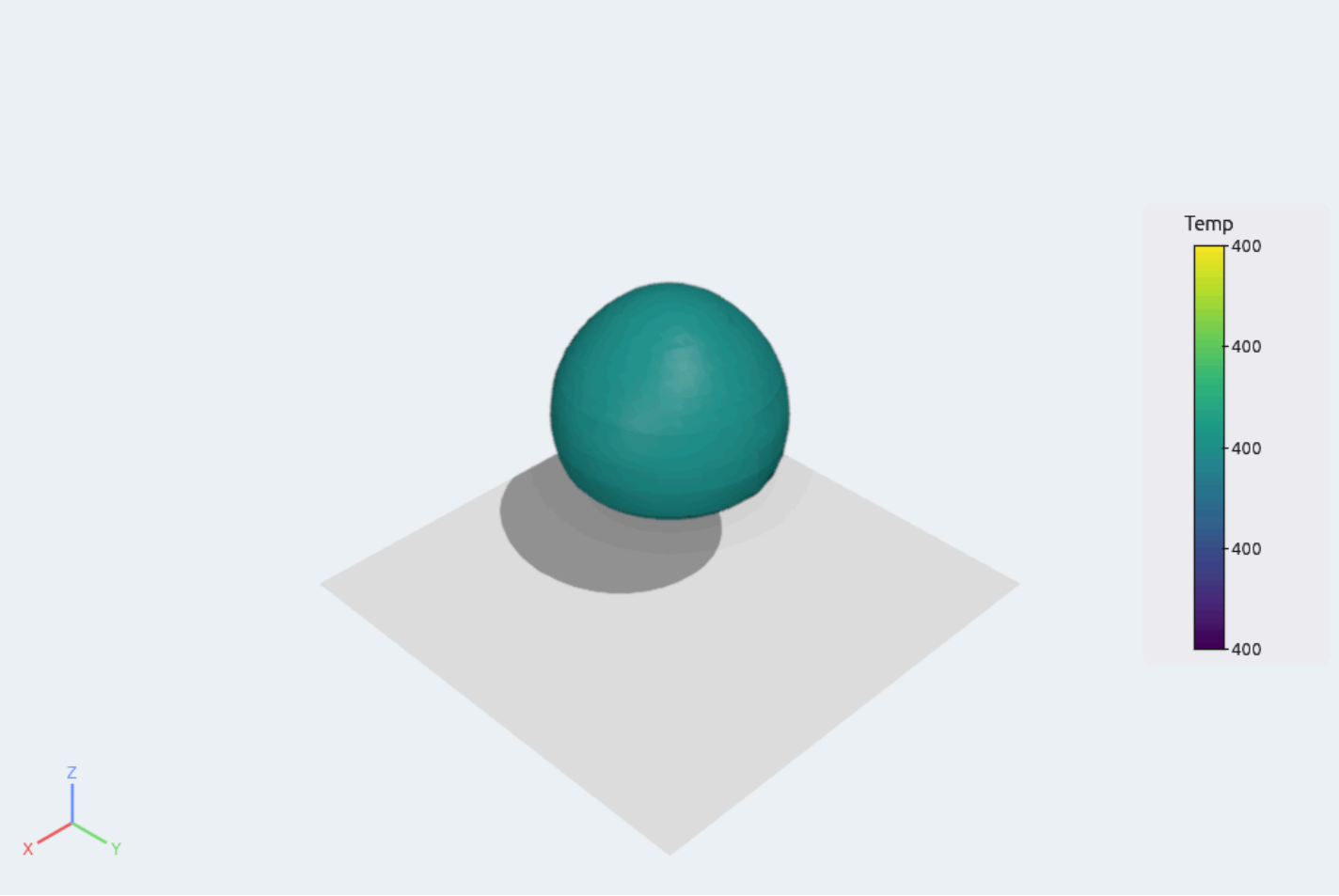
Animations driven by a function

`afx.save_animation("a.mp4", cue=f, frames=60)` calls your function `f(t)` with the animation time `t` from 0 to 1 before each frame (after the keyframes are applied), so anything can change between frames: filter values, colors, the data loaded. The call returns when the file is written.

```
def cue(t):
    c.set(values=[350 + 500 * t])
afx.save_animation("contour_sweep.gif", fps=8, cue=cue, frames=40)
```

Lighting, shadows and effects

Everything here is in **View › Lighting**, and in Python through `afx.lighting(...)`. Calling `afx.lighting()` without arguments returns the current defaults; unknown options raise an error, so typos are caught.



Lights

- **Light kit** (on by default): a key light from the upper right, a fill light from the lower left and a head light, like ParaView's. Untick it for a single head light.
- **Custom lights**: tick it to edit the lights yourself. Each light has a color, an intensity, an azimuth and an elevation, follows the camera or stays fixed in the scene, and can cast shadows. **Add light** adds one; **ambient** sets the light from everywhere.
- **Show lights in the view** draws each custom light as a small sun around the data; drag a sun to aim its light.

```
afx.lighting(light_kit=False)
afx.lighting(lights=[{"direction": (0.5, 0.5, 1), "intensity": 0.8},
                     {"direction": (-0.7, -0.2, 0.5), "intensity": 0.3, "color":
(0.6, 0.7, 1), "camera_relative": False}],
             ambient=0.15)
```

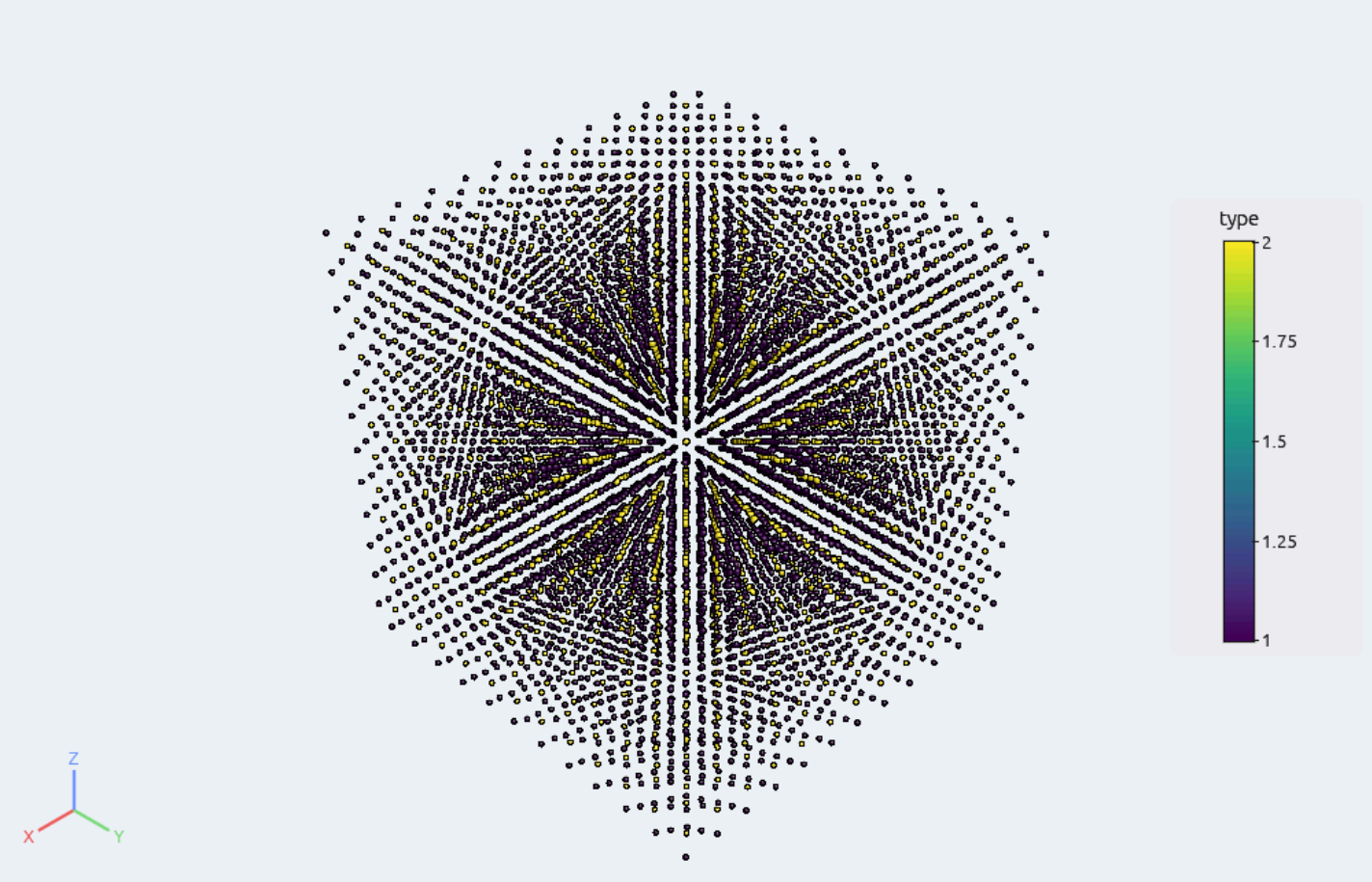
Shadows

Shadows draws the shadow of the key light (or of the first custom light with shadows ticked) on every opaque surface: a shadow map fitted to the visible objects, softened at the edges. Objects drawn by a remote server are shadowed on that machine, so they do not cast shadows on local objects.

Ambient occlusion

Ambient occlusion darkens creases, cavities and the contact between objects, which makes depth easier to read. The sliders set its strength and radius (a fraction of the data size). It works from the picture’s depth, so it costs the same whatever the mesh size.

Eye-Dome Lighting



Eye-Dome Lighting shades every pixel by how much it stands out from its neighbours in depth: outlines and depth cues for point clouds, particles and dense meshes without any normals. The slider sets the strength. `afx.lighting(edl=True)`.

Anti-aliasing, tone mapping and exposure

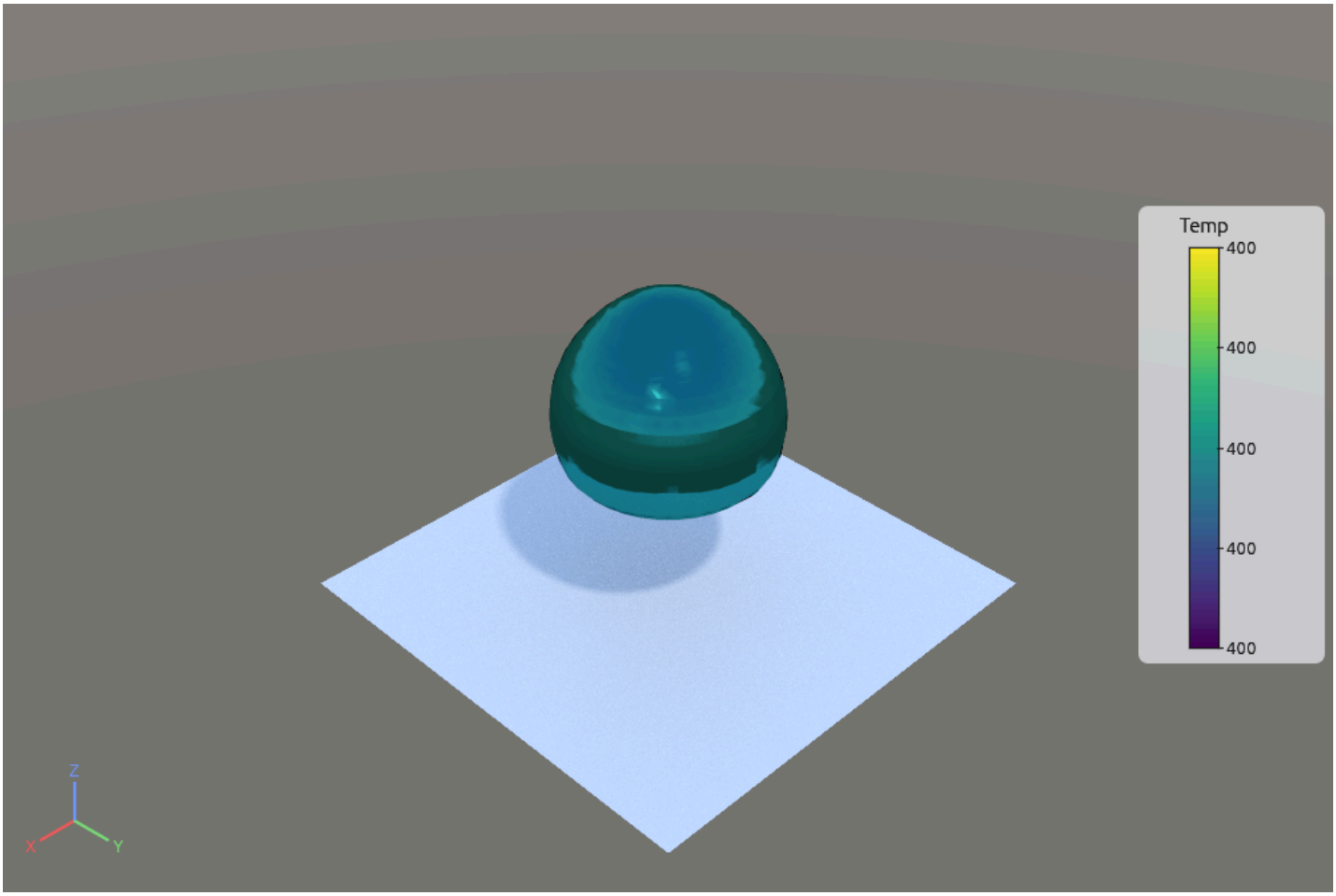
Edges are smoothed by multisampling (4 samples). **FXAA** adds a cheap post pass that smooths the remaining jagged edges of thin lines. **Tone mapping** compresses bright pictures with the **Reinhard** or **ACES filmic** curves, and **exposure** scales brightness before it.

```
afx.lighting(fxaa=True, tone_mapping="aces", exposure=1.2)
```

Transparency

Translucent objects (**Opacity** below 1 in Properties) are blended in a way that does not depend on the order they are drawn in, so several overlapping translucent surfaces look right from any angle. Translucent objects are always drawn on this computer, even when the rest of the data is drawn by a server.

Materials and environment maps



Material in Properties gives each surface a **metal** factor (0 plastic-like, 1 metal: reflections take the object's color), a **rough**ness (small: sharp highlights and mirror-like reflections; 1: matte) and a highlight strength. The defaults reproduce the plain shading.

View › Lighting › Environment lights the scene with a picture of the surroundings: the built-in **sky**, **studio** and **white** environments, or your own equirectangular `.hdr` / `.png` / `.jpg` (z up). It replaces the flat ambient light with soft directional light, adds reflections on shiny and metallic surfaces, and **as background** shows it behind the data. While an environment is on, the built-in lights are dimmed by half.

```
c.set(metallic=1.0, roughness=0.25, specular=0.5)
afx.lighting(environment="sky", environment_intensity=1.2, show_environment=True)
afx.lighting(environment="/data/hdri/studio_small.hdr")
```

Ray tracing

View › Lighting › Ray tracing path-traces the opaque surfaces: soft shadows, light from the sky or environment, and light bouncing between surfaces. It refines while the camera is still (the rasterizer draws while you move) and the status corner shows the samples done. Set the number of **samples** per pixel to stop at, the number of **bounces**, and where it runs: **auto** (the GPU when the data fits in its memory, else the CPU), **GPU** or **CPU**. Edges, points, lines, translucent objects, volumes and the legend are still drawn on top. Screenshots wait for the tracer to finish and are traced at the requested size.

```
afx.lighting(ray_tracing=True, ray_samples=256, ray_bounces=3, ray_device="auto")
afx.screenshot("traced.png", width=3840)
```

Surface LIC



The **Surface LIC** representation (Properties › Representation) draws a vector field as streaks on any surface while the surface stays colored by any array: the classic way to see a flow on a slice. Choose the vector array (**LIC vectors**), the streak **strength** and the **streak length** in pixels.

```
s.set(representation="Surface LIC", lic_vectors="V", color_by="Temp",
lic_length=30)
```

Volumes, point clouds and curved cells

Representations

Representation in Properties (and the toolbar) chooses how an object is drawn:

Representation	Use
Surface	the outer surface, smooth-shaded (hard edges stay sharp: a 30° crease angle)
Surface With Edges	the surface with cell edges (**Edge color** in Properties)
Wireframe	edges only
Points	the points, as flat dots or shaded spheres (**Point size**, **spheres**)
Outline	the bounding box
Volume	GPU ray casting of image data (other meshes are resampled first)
Surface LIC	a vector field as streaks (see Lighting)

Lighting can be switched off per object (flat colors, for 2-D data). **Axes grid** draws a labelled grid around the object with tick values.

Volume rendering



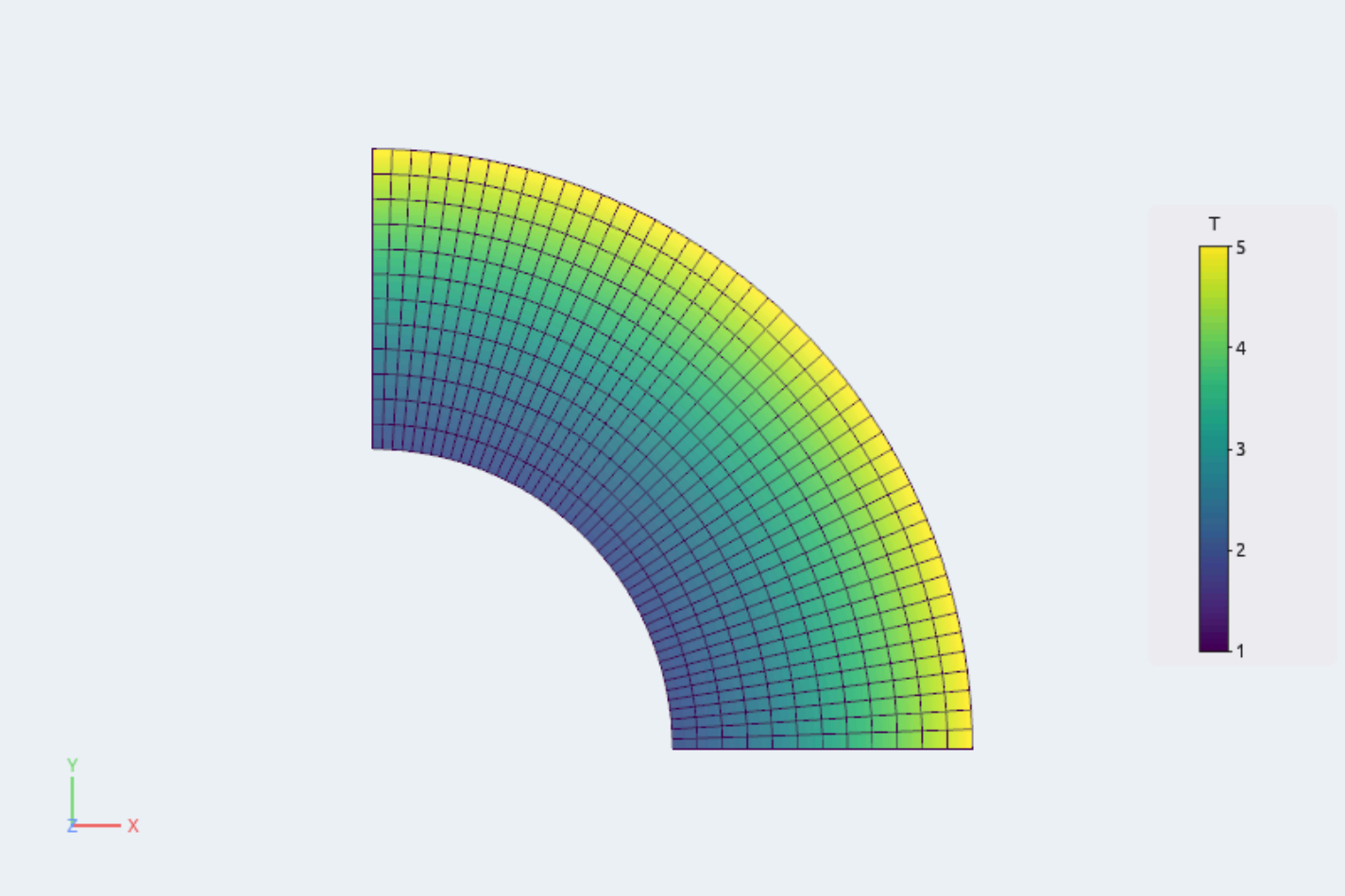
The **Volume** representation draws image data as a translucent cloud colored by the array: **Volume density** sets how opaque it is and **Transparent below** makes values under a fraction of the range invisible, which clears away the noise around a feature. Volumes are depth-merged with the surfaces around them.

```
w.set(representation="Volume", color_by="RTData", volume_opacity=2.0,
volume_threshold=0.3)
```

Point clouds and particles

LAMMPS dumps, XYZ files, PDB files and point sources open as points. Draw them as spheres sized by **Point size**, color them by type, and switch on Eye-Dome Lighting to see the structure of a dense cloud. The **Glyph** filter draws arrows or spheres scaled by an array; **Tube** gives lines a thickness.

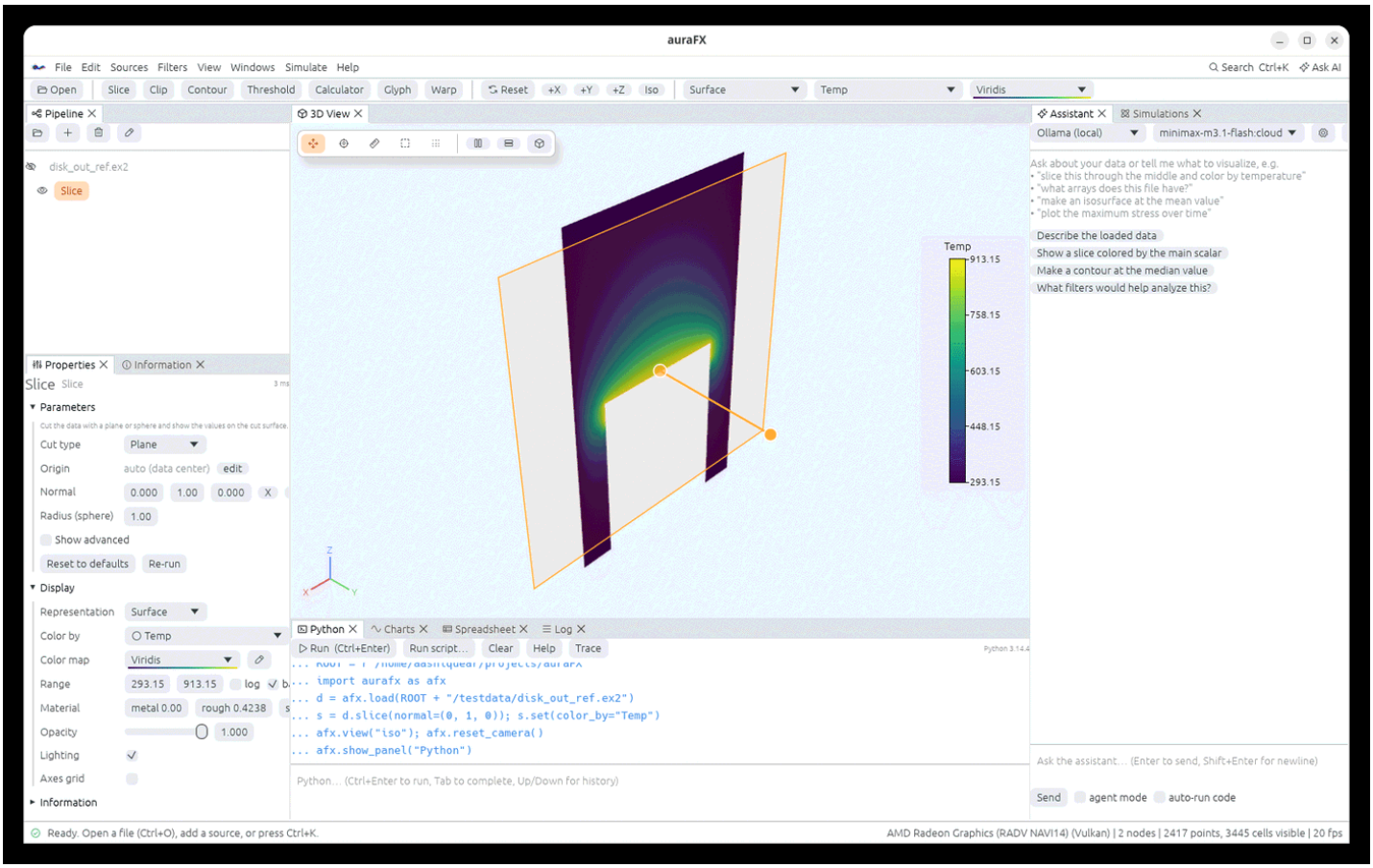
Curved cells



Quadratic cells and Lagrange or Bezier cells of any order (VTK types 68–80, with rational weights and per-cell degrees) are drawn curved. **Curved cells › subdivision** in Properties sets how finely: 0 draws straight edges between corners, 1 passes through the nodes, each further level halves the pieces. Slice, Clip, Contour, Probe and Integrate Variables follow the curved shape, as ParaView does; the **Tessellate** filter turns curved cells into straight ones for filters that need them. The Information panel lists the cell types.

```
a.set(nonlinear_subdivision=3)
t = a.tessellate(level=2)
```

Python



auraFX embeds Python with NumPy. The **Python** panel is a console: type a line and press **Ctrl+Enter** (or **Run**), use Tab to complete, Up/Down for history, **Run script…** to run a file, **Clear** to empty the log, **Help** for the cheat sheet. **Trace** turns what you did with the mouse into the equivalent code, which you can paste into a script. Everything the UI does is reachable from the `afx` module, and `aurafx --script file.py` runs a script at startup (the app stays open afterwards).

Packages installed into the embedded interpreter with `pip` (SciPy, pandas, PyTorch…) are available in the console.

Loading and the pipeline

```
d = afx.load("run.e")                       # a Node; afx.load("slices/img_*.png")
stacks images
d = afx.load("/scratch/run.e", host="cluster")   # read on another machine (see
Data on another machine)
afx.nodes(); afx.active(); afx.get("Slice")      # find nodes
s = d.slice(normal=(0, 1, 0))               # any filter: node.<filter>(**params)
s.set(origin=(0, 1, 0), opacity=0.5)        # parameters and display settings
s.inputs(); s.activate(); s.delete()
s.update(); s.output()                      # wait; the data (an afx.Data: points,
arrays as NumPy)
s.info()                                    # arrays, ranges, bounds, counts
d.array("Temp")                             # a NumPy array; d.add_array("T_C", T -
273.15)
afx.apply_multi("append_datasets", [a, b])
afx.points_from(xyz, temperature=t)         # a node from NumPy
afx.filters(); afx.help_filter("contour")
afx.run()                                   # run whatever is pending
afx.undo(); afx.redo()
afx.save_state("work.afx"); afx.load_state("work.afx")
```

Display

```
d.set(color_by="Temp", cmap="Inferno", range=(300, 900), log_scale=True,
representation="Surface With Edges")
d.set(opacity=0.6, point_size=6, points_as_spheres=True, line_width=2,
axes_grid=True)
d.set(metallic=1, roughness=0.2, specular=0.5)           # material
d.set(representation="Volume", volume_opacity=2, volume_threshold=0.3)
d.set(representation="Surface LIC", lic_vectors="V")
d.set(nonlinear_subdivision=2)                           # curved cells
d.show(); d.hide(); d.set_name("inlet")
```

Views and camera

```
afx.view("iso"); afx.view("+x"); afx.view([1, 1, 0.5]); afx.reset_camera()
afx.split_view("right"); afx.active_view(1); d.send_to_view(1, keep=True)
afx.link_views(True); afx.match_views()
afx.layout("analyze"); afx.show_panel("Charts"); afx.hide_panel("Log");
afx.set_theme("light")
afx.annotate("Run 42", corner="upper left", size=18); afx.time_label(True);
afx.clear_annotations()
afx.lighting(shadows=True, ssao=True, edl=False, fxaa=True, tone_mapping="aces",
             environment="sky", ray_tracing=False)      # see Lighting
```

Time, charts, selection

```
afx.times(); afx.time(10)
afx.over_time(d, "EQPS", stat="max")      # returns (times, values) and plots them
afx.plot(x, y, label="T", title="Temperature"); afx.clear_plots()
afx.temporal_statistics(d)
afx.select(d, ids, "cells"); afx.selection(); afx.extract_selection();
afx.clear_selection()
afx.probe(d, (0, 0, 1))                   # values at a point
```

Pictures, animations, files

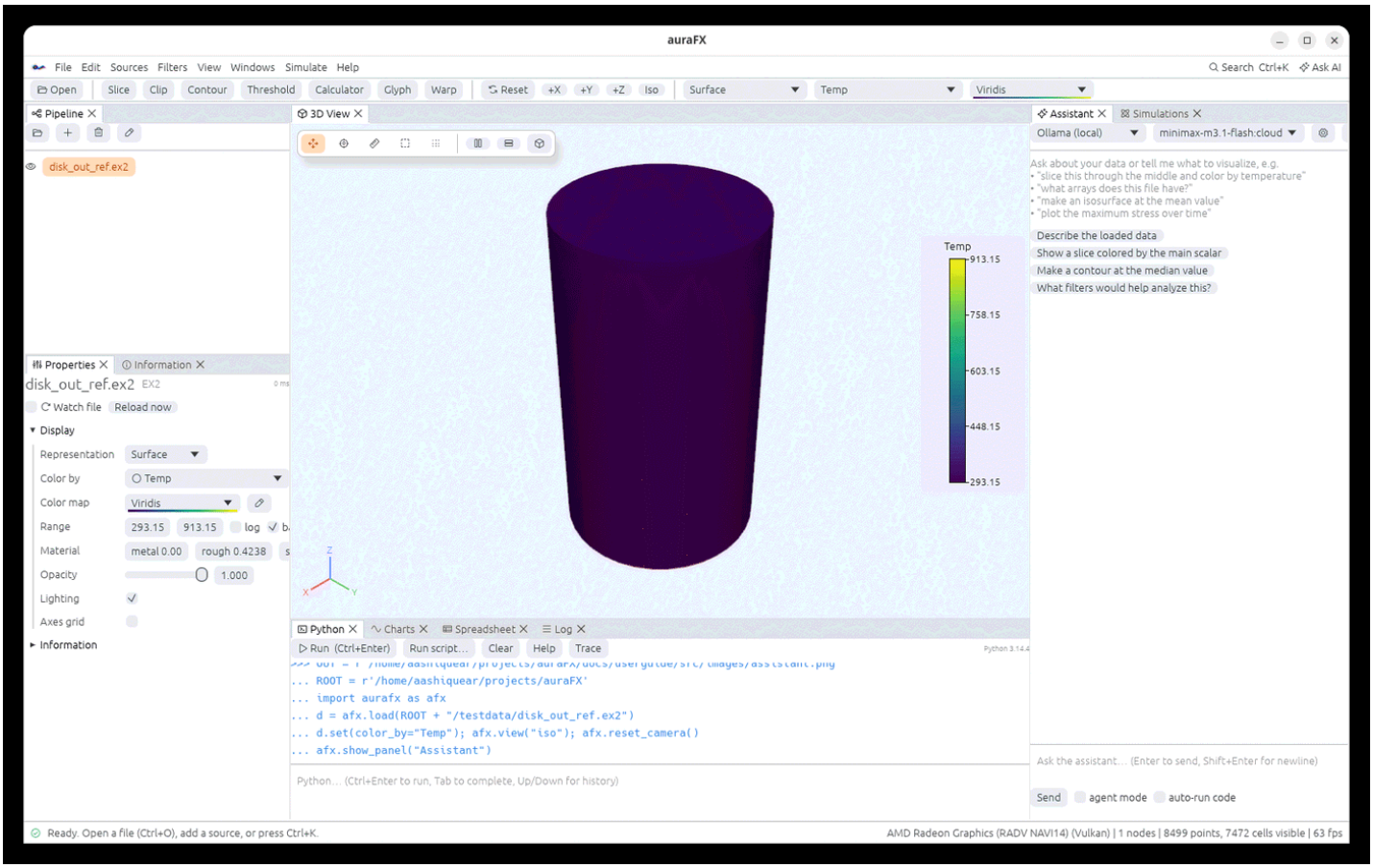
```
afx.screenshot("fig.png", width=3840, transparent=True, annotations=True)   #
waits until written
afx.screenshot("fig.svg")                 # vector picture
afx.export_scene("scene.glb")             # also .obj, .x3d, .wrl
afx.save_animation("run.mp4", fps=24); afx.save_animation("turn.gif", orbit=72)
afx.key_camera(0); afx.key_camera(1, interp="ease in-out"); d.key("opacity", 1,
0.2)
afx.animation(frames=120, time=True); afx.animation_time(0.5);
afx.clear_keyframes()
afx.save_animation("sweep.gif", cue=lambda t: c.set(values=[300 + 600 * t]),
frames=60)
d.save("out.vtu"); d.save("all.exo", all_times=True); d.save("all.vtkhdf",
all_times=True)
afx.colormaps(); afx.define_colormap(name, points);
afx.import_colormaps("maps.json")
```

Assistant, plugins, simulations, servers

```
afx.ask("plot the maximum stress over time", agent=True)
afx.plugins(); afx.load_plugin("tools.py")
afx.submit("lmp -in in.melt", host="rocm-server", workdir="~/runs", gpu="amd:0",
outputs="dump.*", auto_load=True)
afx.sweep("lmp -var T {T} -in in.melt", T=[300, 600, 900], host="cluster")
afx.jobs(); afx.job(id); afx.cancel(id); afx.gpus("rocm-server"); afx.hosts()
afx.servers(); afx.load_remote("cluster", "/scratch/run.e")
```

The **Help › Python API cheat sheet** window lists the same calls inside the app, and every function has a docstring (`help(afx.lighting)`).

AI assistant



The **Assistant** panel (right side; **Windows › Assistant**, or the **Ask AI** button) is a chat with a language model that knows the app and your data. Ask in plain words: “slice this through the middle and color by temperature”, “what arrays does this file have?”, “plot the maximum stress over time”, “why does my contour look cracked?”. The assistant answers and, when something should happen in the app, writes Python using the `afx` API.

What the assistant sees

A text description of your pipeline: the objects, their filters and parameters, array names with ranges, bounds, cell types and time steps, plus the last error. It never receives the data itself. The description is rebuilt for every message.

Running the code it writes

Code appears in a block with a **Run code** button; nothing runs until you press it, unless **auto-run code** is ticked. With **agent mode** on, the assistant runs its code itself, looks at the result (values, errors, the pipeline) and keeps going until the task is done, so “find where the plate yields and plot it” becomes several steps without you in between.

```
afx.ask("make an isosurface at the mean temperature")
afx.ask("find where the plate yields and plot the max stress over time",
agent=True)
```

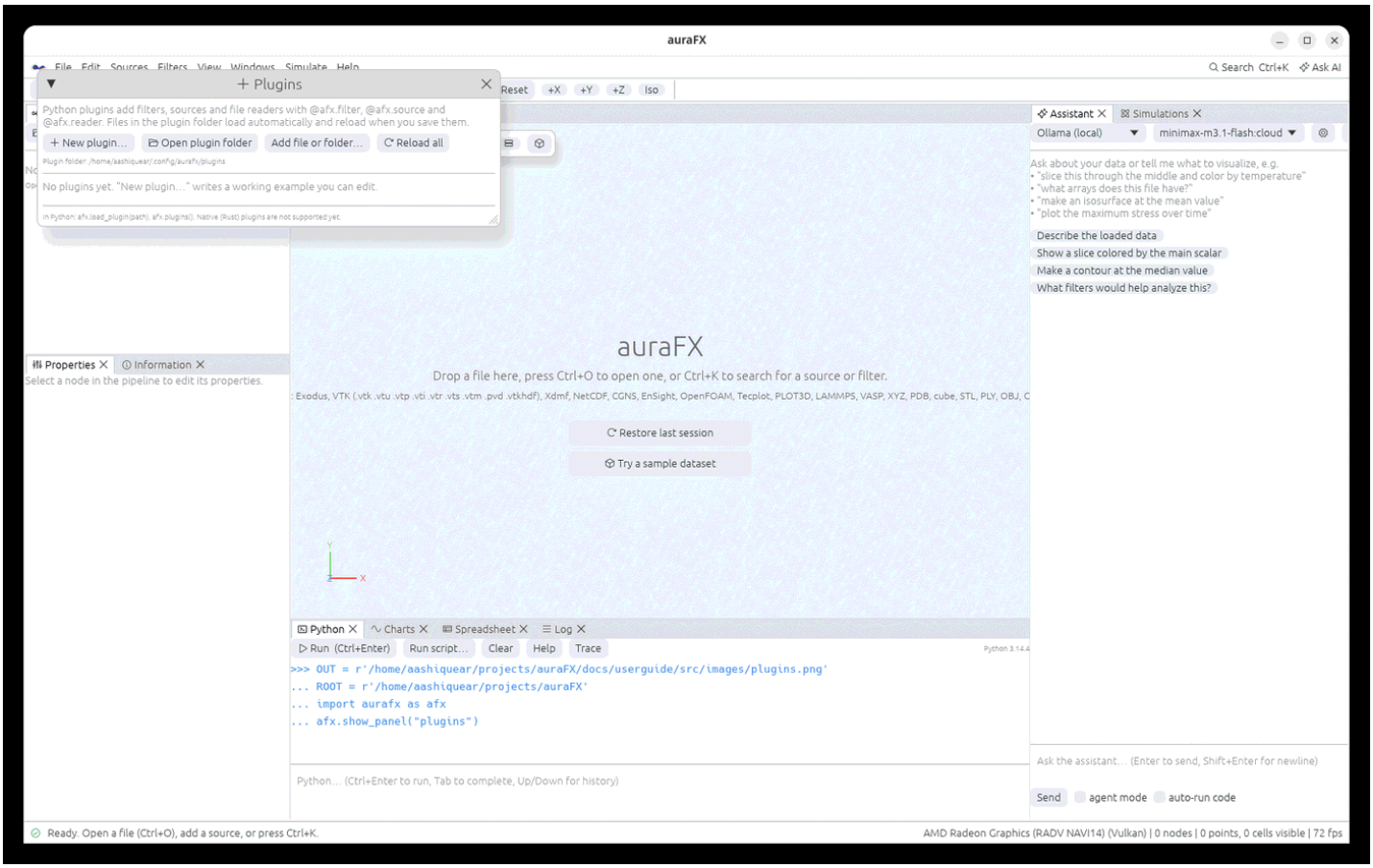
Providers

Choose the provider and model in the panel’s drop-downs or in **Settings**:

Provider	Setup
Ollama (local or cloud models)	the server URL from `OLLAMA_HOST` or Settings; the model list comes from the server
Anthropic	set `ANTHROPIC_API_KEY` or paste a key in Settings
OpenAI-compatible (OpenAI, OpenRouter, Groq, vLLM, LM Studio…)	the base URL and key in Settings

Answers stream as they are produced. The suggestion buttons under the welcome text are good first questions for a new file.

Plugins



A plugin is a Python file that adds filters, sources and file readers. They appear in the Filters and Sources menus, in the command palette, in Python (`node.my_filter()` , `afx.my_source()`), in the Open dialog (readers) and to the AI assistant, like the built-in ones.

Where plugins live

- The plugin folder: **Edit › Plugins… › Open plugin folder** (`~/.config/aurafx/plugins` on Linux, the auraFX config folder on macOS and Windows). Every `.py` file in it is loaded.
- Folders listed in the `AURAFX_PLUGIN_PATH` environment variable.
- Files or folders added with **Add file or folder…** in the Plugins window.
- From a script: `afx.load_plugin("path/to/tools.py")` .

Plugins reload when you save them, and the objects that use their filters re-run. Files whose name starts with `_` are skipped (use them for shared helpers). A plugin that fails to load shows its error in the Plugins window and the Log; switch a plugin off with its checkbox. **New plugin…** writes a working example and opens it in your editor.

A filter

```
import aurafx as afx
import numpy as np

@afx.filter(category="Flow")
def speed(data, vectors=afx.ArrayParam(vector=True), name="speed"):
    """Magnitude of a vector array."""
    v = data.point_data[vectors]
    data.point_data[name] = np.linalg.norm(v, axis=1)
    return data
```

Keyword arguments become parameters in Properties; their type comes from the default: `float`, `int`, `bool`, `str`, a 3-tuple (a vector), `afx.Choice([...])`, `afx.ArrayParam(vector=False, cells=False)` (an array of the input) and `afx.Code("...")` (a text box). The docstring's first line is the description. Other decorator options: `label`, `description`, `keywords`, `id`, and `inputs=2` or `inputs=(2, None)` for filters with several inputs (the function receives a list).

`data` is an `afx.Data`, a copy of the input: `data.points` (N×3), `data.point_data["T"]`, `data.cell_data[...]`, `data.field_data[...]` (assign to add, `del` to remove), `data.n_points`, `data.n_cells`, `data.bounds`, `data.type`, `data.cells` (offsets, connectivity, VTK types), `data.blocks()`, `data.copy()`. ParaView spellings such as `data.PointData.append(a, "name")` work too. Build new data with `afx.make_points`, `afx.make_polydata`, `afx.make_image`, `afx.make_grid` and `afx.make_multiblock`.

Sources and readers

```
@afx.source(category="Shapes")
def helix(turns=4.0, points=400):
    t = np.linspace(0, turns * 2 * np.pi, int(points))
    return afx.make_polydata(np.c_[np.cos(t), np.sin(t), t / 10], lines=
[list(range(len(t)))])

@afx.reader(extensions=["xyzt"], label="XYZ + temperature")
def read_xyzt(path):
    a = np.loadtxt(path, ndmin=2)
    d = afx.make_points(a[:, :3])
    d.point_data["T"] = a[:, 3]
    return d
```

Readers take precedence over the built-in readers for their extensions.

Rules

- Plugin functions run on a worker thread while the pipeline runs: do not call pipeline functions (`afx.load`, `node.slice()`, `show`…) inside them.
- Ids must be unique and cannot replace built-in filters.
- For one-off code, the **Programmable Filter**, **Programmable Source** and **Python Calculator** run Python stored in the pipeline (and in state files).
- Native (Rust) plugins are not supported yet.

File formats

Every reader was checked against ParaView 6.1.1: the same points, cells, bounds and value ranges on the same files. Where auraFX differs on purpose, the table says so.

Readers

Format	Extensions	Notes
Exodus II	`.e .exo .ex2 .g .gen`	NetCDF-3 and NetCDF-4; blocks, side and node sets, nodal/element/global variables, time steps, displacements
VTK legacy	`.vtk`	ASCII and binary, all dataset types
VTK XML	`.vtu .vtp .vti .vtr .vts .vtt`, parallel `.pvtu …`, `.vtm`, `.pvd`	inline, appended, base64; zlib and LZ4; curved (Lagrange/Bezier) cells
VTKHDF	`.vtkhdf .hdf`	image data, unstructured grids, poly data, partitioned collections with their assembly, time steps; chunked and gzip storage; no HDF5 library needed
OpenFOAM	`.foam`	ASCII/binary/gzip, decomposed and reconstructed cases, time directories, boundary patches
EnSight Gold	`.case .encas`	ASCII and C binary, all element types, structured blocks, time sets, wildcard file names
CGNS	`.cgns`	HDF5 storage; structured and unstructured zones, MIXED and NGON/NFACE sections, solutions, time steps; ADF storage is not read
Xdmf 2/3	`.xmf .xdmf`	inline, HDF5 and raw binary data; collections; `xi:include` and XPath references (FEniCS/Dolfinx); HyperSlab and JOIN items
NetCDF (CF)	`.nc .nc4 .cdf`	classic and NetCDF-4; lon/lat grids on a sphere with cell bounds, rectilinear

Format	Extensions	Notes
		otherwise; time steps; packing; fill values become NaN (grey)
Tecplot	`.dat` `.plt` `.tec` `.tp`	ASCII (ordered and FE zones, BLOCK/POINT, cell-centered variables) and binary `.plt`; `.szplt` is not read
PLOT3D	`.xyz` `.x` `.p3d` `.bin` + `.q`	layout detected from the file (Fortran/C binary, byte order, precision, multi-grid, 2-D/3-D, IBLANK), ASCII
LAMMPS	`.dump` `.lammpstrj` `.atom` (+ `.gz`), `.data` `.lmp`	multi-frame dumps with any columns, triclinic boxes; data files with bonds
XYZ / extended XYZ	`.xyz` `.extxyz`	multi-frame; properties and lattice from the comment line
PDB	`.pdb` `.ent`	all atoms (hydrogens kept), MODELs as time steps, CONECT and distance bonds, residues and chains
Gaussian cube	`.cube`	the density as a volume in Å, plus the atoms with bonds
VASP	`POSCAR`, `CONTCAR`, `CHGCAR`, `LOCPOT`, `ELFCAR`, `PARCHG` (by name), `.vasp`	atoms with bonds and the lattice; volumes on the cell's axes (charge per Å³)
Images	`.png` `.jpg` `.bmp` `.tif`	as image data; multi-page TIFFs and `img_*.png` wildcards as volumes
Surfaces	`.stl` `.ply` `.obj`	
CSV / TSV	`.csv` `.tsv` `.txt`	delimiter and header detection; tables with x, y, z columns become points

Numbered files (`run_0001.vtu` …) open as a series when one of them is opened.

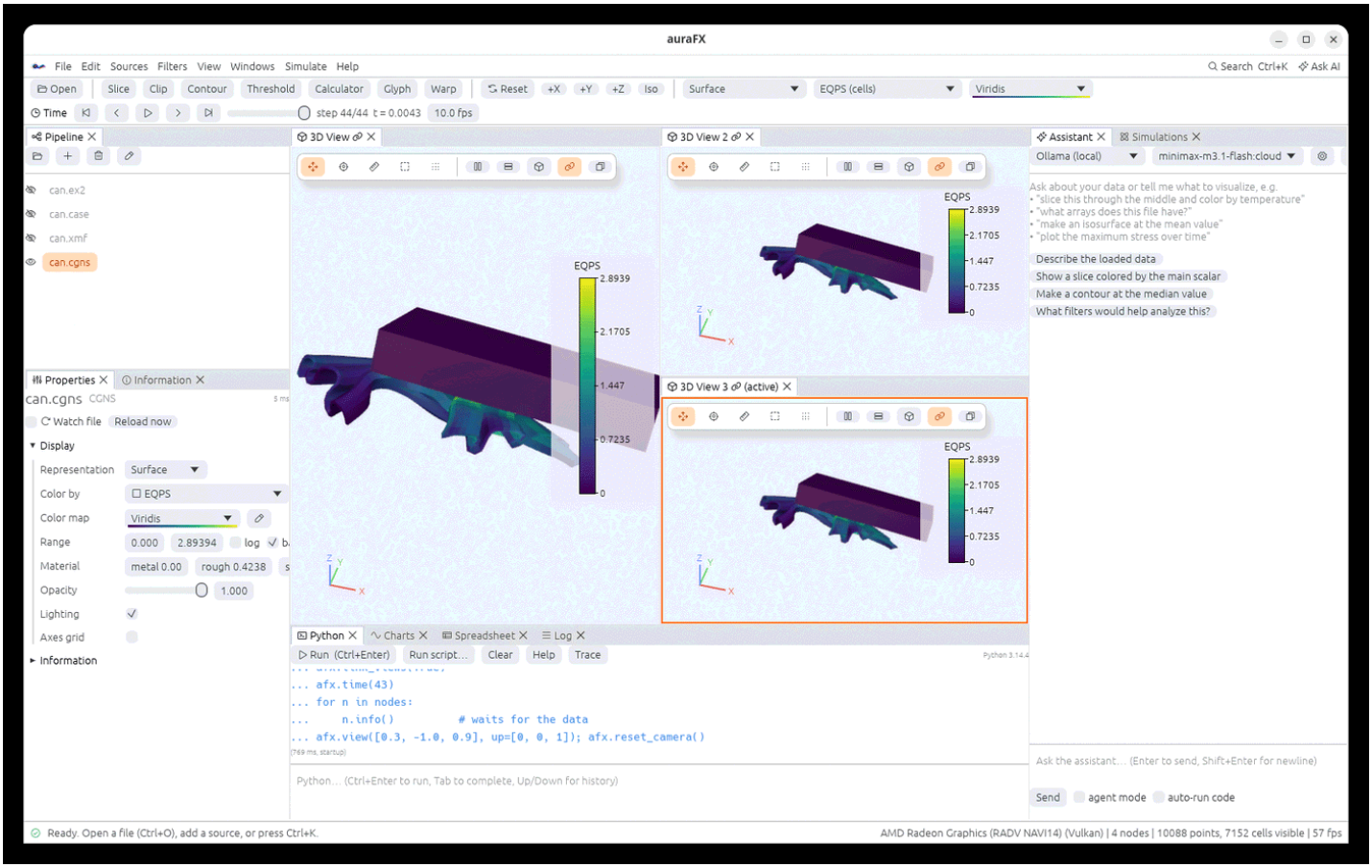


Writers

Format	Extensions	Notes
VTK XML	`.vtu` `.vtp` `.vti` `.vtr` `.vts` `.vtm`	appended binary; multiblocks as a folder of pieces
VTK legacy	`.vtk`	
VTKHDF	`.vtkhdf`	all dataset kinds; multiblocks as a collection; time series with `all_times=True`; written with auraFX's own HDF5 encoder (read by h5py and VTK)
Exodus II	`.exo`	blocks, nodal and element variables, every time step with `all_times=True`
EnSight Gold	`.case`	C binary; one part per block (image, rectilinear and structured grids as blocks, the rest as elements of every standard type, polygons and polyhedra); every time step with `all_times=True`, the geometry written once when it does not change

Format	Extensions	Notes
CGNS	`.cgns`	HDF5 storage (CGNS 4): a zone per block, image/rectilinear/structured grids as structured zones, the rest as one element section (MIXED when the cell types differ) plus NGON/NFACE for polygons and polyhedra; vectors as `NameX/NameY/NameZ`; with `all_times=True` one solution per step (and coordinates per step when the points move), but the cells must stay the same
Xdmf 3	`.xmf` + `.h5`	the description in the `.xmf`, the arrays in an HDF5 file beside it; block trees as spatial collections, time steps as a temporal collection that shares unchanged points and cells
CSV, STL, PLY		
Scenes	`.glb` `.obj` `.x3d` `.wrl`	see Exporting
Pictures	`.png` `.svg`, GIF/MP4 animations	see Exporting

Files written by the VTK XML, VTKHDF, Exodus, EnSight, Xdmf and CGNS writers open in ParaView with the same points, cells and values. EnSight groups the cells of each part by type, and CGNS puts polyhedra after the other cells, so cells can come back in a different order (each keeps its values). Cell types a format does not have (Lagrange and Bezier cells, for example) are written by their corner points. A CGNS zone holds cells of one dimension: in a volume mesh, boundary faces and lines are left out.



Curved cells

VTK's Lagrange and Bezier cells of any order, quadratic cells and serendipity cells are read with their node numbering, drawn curved and cut by the filters along their curvature; see Volumes, point clouds and curved cells.

Your own formats

A plugin reader is a Python function with `@afx.reader(extensions=[...])`; it takes precedence over the built-in reader for its extensions and appears in the Open dialog.

Exporting pictures, scenes and animations

Screenshots

File › Save Screenshot… (Ctrl+Shift+S) opens a small dialog:

- **Size**: the view's size, a multiple of it (2×, 4×), 1920×1080, 3840×2160, or any size; **keep the view's shape** ties width and height together. Point sizes and line widths are in pixels, so they look thinner in a larger picture.
- **Background › transparent** saves a PNG with an alpha channel, with clean edges.
- **Annotations** includes the legend, axes-grid labels, time label, text annotations and the orientation axes, laid out as on screen and scaled with the picture.
- **PNG** or **SVG**: an SVG holds the surfaces as shaded polygons sorted back to front, the lines and points, and the annotations as real text, for figures that scale without pixels (where surfaces cut through each other the boundary is jagged; convert to PDF with Inkscape if needed).

With ray tracing on, the picture is traced at the requested size first.

```
afx.screenshot("fig.png")                                  # the view as shown
afx.screenshot("fig.png", width=3840, transparent=True) # keeps the view's shape
afx.screenshot("fig.png", width=1600, height=900, annotations=False)
afx.screenshot("fig.svg")
```

`afx.screenshot` returns only when the file is written, so a script can change the camera or the data between pictures.

Scenes for other programs

File › Export Scene… (`afx.export_scene("scene.glb")`) writes the visible objects, with their colors, as:

Format	Notes
glTF binary (`.glb`)	for Blender, three.js, PowerPoint, Windows 3D Viewer; vertex colors, normals, translucency, materials (metallic, roughness)
OBJ (`.obj`)	with vertex colors after the coordinates (Blender and MeshLab read them)

Format	Notes
X3D (`.x3d`), **VRML** (`.wrl`)	absolute coordinates, vertex colors; ParaView reads the VRML

Volumes are not exported (they are not geometry).

Animations

File › Animation… records the time steps, a turntable or the keyframes as **GIF**, **MP4** (`ffmpeg` on the PATH) or a folder of PNG frames; see Time, charts and animation.

Saving data

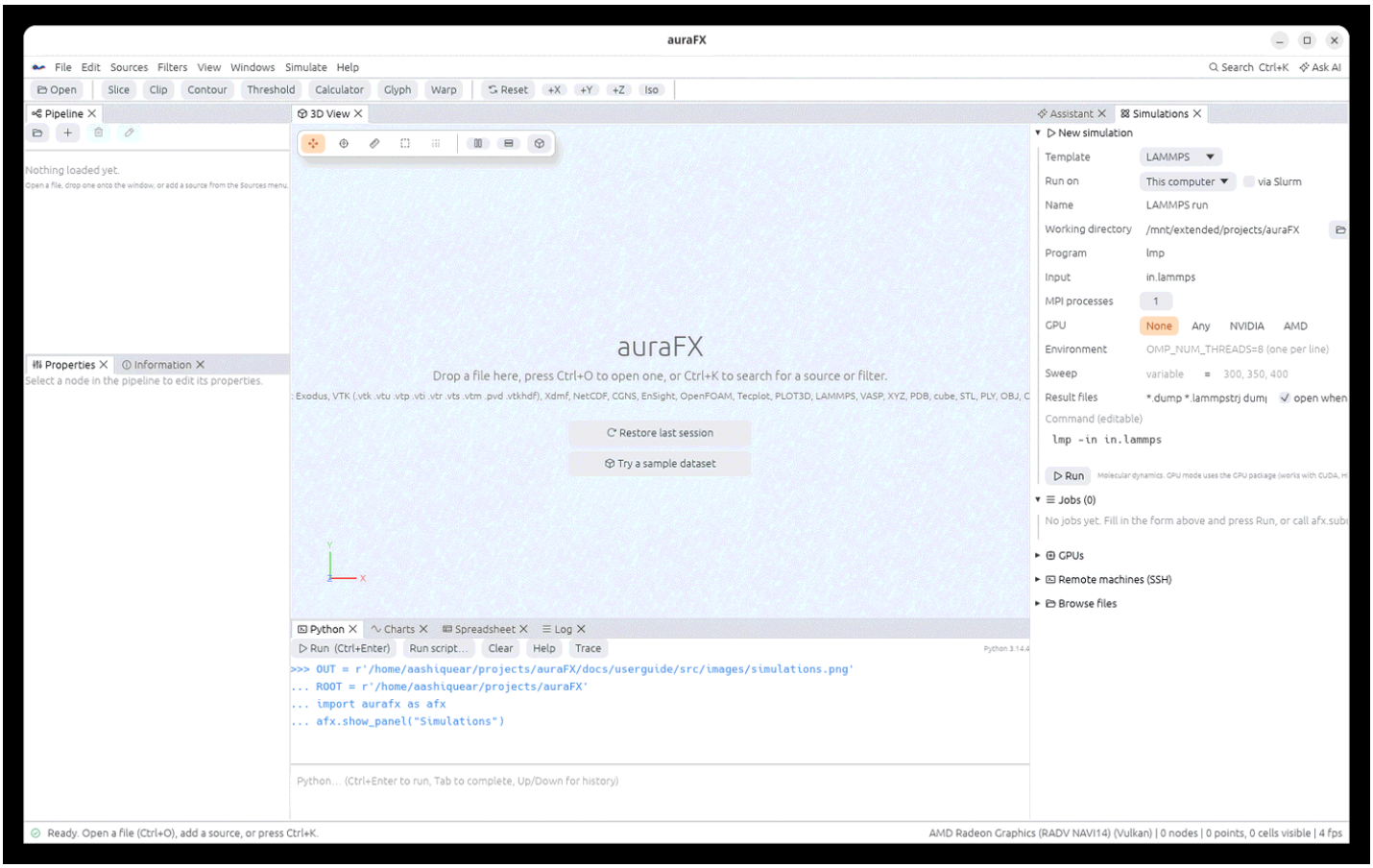
Right-click an object › **Save data…**, or `node.save("out.vtu")` , writes its output; the format follows the extension:

Extension	Format
`.vtu .vtp .vti .vtr .vts .vtm`	VTK XML (appended binary); `.vtm` writes a folder of pieces
`.vtk`	legacy VTK
`.vtkhdf`	VTKHDF (HDF5); multiblocks as a collection with the block tree; `all_times=True` writes every time step into one file
`.exo`	Exodus II with blocks and variables; `all_times=True` writes every time step
`.case`	EnSight Gold (C binary): the case file plus a geometry file and one file per variable next to it; `all_times=True` writes every time step
`.xmf`	Xdmf 3: the description plus an `.h5` file with the arrays; `all_times=True` writes every time step
`.cgns`	CGNS (HDF5); `all_times=True` writes every time step when the cells stay the same (moving points are fine)
`.csv`	the point data or the table
`.stl .ply`	surfaces

```
d.save("slice.vtp")
d.save("run_all.exo", all_times=True)
d.save("run_all.vtkhdf", all_times=True)
d.save("run.case", all_times=True)      # run.case, run.geo, run.Temp0000, ...
d.save("run.xmf", all_times=True)       # run.xmf and run.h5
d.save("run.cgns", all_times=True)
```

When the data has several time steps and the chosen format holds them all (Exodus, VTKHDF, EnSight, Xdmf, CGNS), the Save dialog writes every step.

Running simulations



The **Simulations** panel (right side; **Simulate › Run a simulation…**, or **Windows › Simulations**) launches solvers and opens their results when they finish.

A new simulation

Field	Meaning
Template	LAMMPS, MOOSE app, OpenFOAM, Python pipeline or Custom command; fills in the program, input and result-file patterns
Run on	this computer, or a remote machine from the SSH list; **via Slurm** submits with `sbatch` there (partition, time and extra `#SBATCH` lines appear)
Name, **Working directory**	where the run lives; a remote directory when running over SSH
Program, **Input**, **MPI processes**	the command line is assembled from these and shown below, editable

Field	Meaning
GPU	None, Any, NVIDIA or AMD; the panel finds the GPUs on the host and sets the solver's flags (LAMMPS GPU package, `CUDA_VISIBLE_DEVICES` / `HIP_VISIBLE_DEVICES`)
Environment	extra `VAR=value` lines (one per line)
Sweep	a variable name and a list of values: one job per value, `{var}` in the command replaced, each in its own folder
Result files	patterns of the files to open when the run ends (**open when done**)

Run starts the job. Jobs appear in the **Jobs** list with their state, a live log tail, and buttons to cancel, open the results or the folder, and resubmit. Files a running solver writes can be opened while it runs; auraFX reloads them as they change (live reload).

Remote machines

Remote machines (SSH) holds the hosts: a name, the SSH target (`user@machine`, or an alias from `~/.ssh/config`), port, key file, jump host, and **setup lines** run before every command (module loads, `source` of an environment, `AURAFX_SERVER=...`). SSH keys or an agent must be set up so that `ssh host` works without a password prompt. The same hosts serve **File › Open on Server...** (see Data on another machine).

GPUs lists the GPUs of each host; **Browse files** opens a remote file browser to pick inputs or results.

From Python

```
jid = afx.submit("lmp -sf gpu -pk gpu 1 -in in.melt", host="rocm-server",
                 workdir="~/runs/melt", gpu="amd:0", outputs="dump.melt",
auto_load=True)
afx.sweep("lmp -var T {T} -in in.melt", T=[300, 600, 900], host="cluster",
workdir="~/sweep")
afx.jobs(); afx.job(jid); afx.cancel(jid)
afx.gpus("rocm-server"); afx.hosts()
afx.load_remote("rocm-server", "~/runs/melt/dump.melt")   # copy a file here and
open it
```

`gpu` is `None`, `"any"`, `"nvidia"`, `"amd"` or e.g. `"nvidia:0,1"`. `afx.job(id)` returns the state, exit code, log tail and output files.

Data on another machine

When the data lives on a cluster or a GPU box, auraFX can read and filter it **there** and bring back only what you look at, like ParaView's `pvserver`, with nothing to set up beyond SSH.

How it works

`aurafx-server` is a small program without a window. The app starts it on the host over SSH (`ssh host aurafx-server --stdio`) and talks to it through that connection, so keys, jump hosts and `~/.ssh/config` work as usual and no port is opened. Files you open on the server are read there; filters you add below them (Slice, Clip, Contour, Threshold, Stream Tracer, temporal filters and so on) run there too. What comes back:

Situation	What is transferred
Visible, output smaller than 64 MB	the whole output
Visible, larger	its outer surface (all arrays), which is what the 3D view draws
Hidden (for example a reader with a slice below it)	only its summary: arrays, ranges, bounds, time steps
Bring all data ticked in Properties, or a filter below it that runs here	the whole output

The Information panel always describes the full data on the server. Python filters (Programmable Filter, Python Calculator) and plugin filters run on this computer, on the data brought over. The server keeps results, so changing one filter only re-runs that filter there.

Setting up a host

1. Make sure `ssh yourhost` works without a password prompt.
2. Put `aurafx-server` on the host: **File › Open on Server… › Install server** copies the one shipped with the app to `~/.local/bin` (Linux hosts of the same architecture; the packages run on glibc 2.35 or newer), or build it there with `cargo build --release -p aurafx-server`, or point `AURAFX_SERVER` at it in the host's setup lines.
3. **Check server** shows its version and GPU.

Hosts come from the Simulations panel and from `~/.ssh/config`. “This computer” runs the server locally, which is handy for trying it out.

Opening files

File › Open on Server…: pick the host, browse to the file, double-click it. A cloud icon marks objects that run on a server; hover it, or look at the top of Properties, to see what was transferred and how long the server took.

```
d = afx.load("/scratch/run42/out.e", host="cluster")   # a Simulations host, ssh
alias or user@machine
d = afx.load("ssh://cluster/scratch/run42/out.e")       # the same
s = d.slice(normal=(0, 0, 1))                            # runs on the server
s.set(all_data=True)                                     # bring the whole output
over
afx.servers()                                            # connections and the
machines' details
```

Drawing on the server

Some outputs are too big even as a surface. Then the server draws the object with its own GPU and sends pictures (a color and a depth image per view, compressed), which the 3D view blends by depth with the local objects, so everything hides and is hidden correctly. While you move the camera the last picture stays up until the next one arrives.

Display › Draw on in Properties picks per object: **auto** (drawn here unless the surface is over 64 MB), **this computer**, or **the server** (`node.set(draw_on="server")`). Picking, rulers and selections need the geometry here, so switch to “this computer” for those. Translucent objects, volumes and Surface LIC are always drawn here. The view’s lights and shadows travel with each request; screen effects (ambient occlusion, Eye-Dome Lighting, FXAA, tone mapping) are applied here to the whole picture.

Parallel servers

Data too big for one process can be spread over several, like `mpirun pvserver` : **Open on Server › Processes**, or `afx.load(path, host="cluster", processes=8)` , starts eight worker processes on the host (`aurafx-server --workers node1:16,node2:16` spreads them over machines sharing a file system). Partitioned VTK files are read piece by piece; other files are

read by every process and split. Cell-by-cell filters run on every piece with automatic values taken from the whole data; the pieces come back appended, as a merged surface, or as pictures composited by depth. Results match a single process exactly.

Not yet: MPI launchers (`srun`) for the workers, ghost cells, sharing one server between people, temporal filters on parallel servers.

Settings and keyboard shortcuts

Settings

Edit › Settings… (Ctrl+,):

Setting	Meaning
Theme	dark, light or system; the 3D background per theme
UI scale	enlarges the whole interface
Orientation axes, **Time label**	shown in the 3D views
Filter threads	cores used by the parallel filters (0 = all); takes effect at the next start; `AURAFX_THREADS` overrides it
Default color map	for new objects
AI	provider, model, server URL, API key, auto-run and agent mode
Plugins	the Plugins window (also **Edit › Plugins…**)
Lighting	in **View › Lighting**; the choices are saved with the settings

Settings live in `~/.config/aurafx/settings.toml` (Linux) or the app's config folder on other systems; the window layout in `~/.local/share/aurafx/app.ron`.

Keyboard shortcuts

Keys	Action
Ctrl+O	open a file
Ctrl+S	save state
Ctrl+Shift+S	save a screenshot
Ctrl+K, Ctrl+Space	command palette
Ctrl+Z, Ctrl+Y	undo, redo
Ctrl+Enter	run the code in the Python console
Ctrl+,	settings

Keys	Action
R	reset the camera
Esc, P, M, S, D	view modes: navigate, pick, ruler, select cells, select points
Delete	delete the selected object
Space	play / pause time
Double-click in the view	fit the data

Help › Keyboard shortcuts shows this list in the app.

Environment variables

Variable	Use
`AURAFX_THREADS`	thread limit for filters
`AURAFX_PLUGIN_PATH`	extra plugin folders
`AURAFX_SERVER`	path of `aurafx-server` on a host (in its setup lines)
`AURAFX_DATA_DIR`	where settings and state are kept
`OLLAMA_HOST`, `ANTHROPIC_API_KEY`	AI providers
`PYO3_PYTHON` (at build time)	which Python to embed